Active Learning for Binary Rail Segmentation — Experiment Report

This document is a self-contained record of **two phases** of active-learning experiments on the same binary rail-segmentation task. It is written so that it can be pasted into a fresh chat as full context.

- **Phase 1 (local, May 14 – May 28 2026).** Six AL strategies + fulldata upper bound, **n=3 seeds-of-noise** per strategy, plus two ablations (`coreset_only_old_data` n=3, `fulldata_old_only` n=1) and a separate **low-budget** experiment (`random` and `bvsb` over 15 cycles, n=3 seeds). Trained at **BATCH_SIZE=4**.
- **Phase 2 (server `Ubuntu-Devbox`, May 27 – Jun 10 2026).** Same six strategies **plus three new ones** (`power_bvsb` , `bvsb_coreset` , `badge`) and fulldata, **n=10 seeds-of-noise** per strategy, **BATCH_SIZE=8** (main sweep; the FixMatch notebook stays at labeled batch 4 — see §9.6). Plus **n=5 FixMatch SSL** runs (BvSB-only, small budget, 8 cycles) that probe Mittal-style "does AL beat random under SSL?" — the matched random baseline was deliberately deferred.

Code and aggregation:

- Main notebook: `train_clearml.ipynb` (working copy `train_clearml_copy.ipynb`).
- FixMatch notebook: `fixmatch/nb_low_budget_fixmatch.ipynb` .
- Phase 1 aggregator: `plot_runs_comparison.py` → outputs in this directory (`plots_comparison/`).
- Phase 2 aggregator: `/media/samsepi/data/server_runs/plot_server_runs.py` → outputs copied into `plots_comparison/server_runs/` for portability.

---

1. Task & motivation

Binary semantic segmentation of **rails** (class 1) vs background (class 0) on aerial imagery. The motivating question:

> *Given a fixed labeling budget, which active learning strategy picks the most informative images to label? And does any of them recover the performance of training on the full dataset?*

Phase 2 sharpened three secondary questions that Phase 1 left open:

> *With more seeds (n=10 vs n=3), does the n=3 ranking survive? Do diversity-aware strategies (BADGE-style) help, or does pure uncertainty dominate? Does adding an SSL objective (FixMatch) close the AL-vs-random gap in the low-budget regime?*

---

2. Dataset construction

There are **two sources of data** and the way they're combined is critical to interpreting the results.

2.1 The two pools

pool	path	label quality	typical content	size on disk
`real`	`/media/samsepi/data/datasets/results_test_clean`	**pseudo-labels** (output of a prior model on field imagery), so noisy / not clean ground truth; acquired by fine-tuned DinoV3-SAT493M (frozen 7B DinoV3 with deep adapter trained on `old`)	"real" field photos — variable lighting, angles, occlusions	1980 images
`old`	`/media/samsepi/data/datasets/tiled_dataset_rails`	**clean human/curated labels**	curated drone tiles (DJI source), uniform conditions	train 3321, valid 989, test 455

Phase 2 datasets on the server live under `/workspace/ultralytics1/SyzdykovN/{results_test_clean,tiled_dataset_rails}` with identical contents (verified by image count: 1980 / 3321 / 989 / 455).

2.2 Training pool (`CombinedDataset`)

The training pool used by all strategies is a 50/50 mix of `real` and `old`:

- All 1980 `real` images (full set).
- A random subsample of 1980 `old` images drawn from the 3321 available (seeded for reproducibility — `DATASET_SEED = 96`).
- **Total training pool: 3960 images**, with indices 0..1979 being `real` and indices 1980..3959 being `old`. The dataset exposes a `.source(idx)` method that returns `'real'` or `'old'` for AL pick-composition logging.

2.3 Validation and test sets

Both come from `tiled_dataset_rails` exclusively — i.e. **clean labels only**:

- **Validation:** 989 images from `tiled_dataset_rails/valid`.
- **Test (holdout):** 455 images from `tiled_dataset_rails/test`.

This is a deliberate methodological choice and a key confound to keep in mind: **the training pool is 50% noisy pseudo-labels, but the test set is 100% clean labels.** Any strategy that preferentially picks pseudo-labelled samples is training on a distribution that doesn't match what it's evaluated on.

2.4 Image / mask handling

- Images opened as RGB. Masks opened as L (grayscale), then binarized via `> 128` after augmentations (Albumentations pipeline followed by `ToTensorV2`).
- Image size: **1024 × 1024** (downscaled / cropped from native).
- Batch size: **4 (Phase 1) → 8 (Phase 2 main)**. The change to 8 in Phase 2 is intentional (faster wall-clock per cycle on the server's 3090) but means Phase 2 absolute numbers are not drop-in replacements for Phase 1; see §9.6. **Exception:** the Phase 2 FixMatch notebook stays at labeled batch 4 (§4.5, §9.6).

2.5 Augmentations (train only)

Implemented via `HybridTransformWrapper` wrapping an Albumentations pipeline. Unchanged between Phase 1 and Phase 2.

- Color: `ColorJitter` (mild), `ToGray` (p=0.1), `GaussianBlur` (large kernel).
- Geometry: `OneOf(heavy_geometry, resize_or_crop)`:
- `heavy_geometry`: `OneOf(RandomScale zoom-out, RandomResizedCrop zoom-in)`
  - horizontal/vertical flips.
- `resize_or_crop`: 70% plain `Resize`, 30% `CropNonEmptyMaskIfExists` (native 1:1 crop around rails when present).
- `RandomLineOcclusion` (custom): draws thin random lines over the image but not the mask, to teach the model to "complete" rails under occlusions (sticks, wires, shadows).
- `ToFloat(max_value=255.0)` + `ToTensorV2`.

Validation transforms are just `Resize` + `ToFloat` + `ToTensorV2` (no augmentation).

The **FixMatch notebook** (Phase 2) uses these transforms only on labeled images. The unlabeled weak/strong paired pipeline is defined separately in `fixmatch/fixmatch.py`; see §4.5.

---

3. Model

`smp.DeepLabV3Plus` with a few customisations (identical across both phases):

- Encoder: `resnet34`, `imagenet` pretrained, `encoder_output_stride=16`.
- Output: 2-class softmax head (background, rail).
- **Extra MC-Dropout layers** (channel-wise `Dropout2d`, p=0.1) inserted at three feature locations to make BALD informative:
- `mc_drop_deep` on the deepest encoder feature (input to ASPP).
- `mc_drop_skip` on a high-res encoder feature (skip into decoder).
- `mc_drop_dec` on the decoder output before the segmentation head.
- A `forward(x, return_embedding=True)` mode returns mean-pooled deepest encoder features (shape `[B, C]`) **without** dropout, for Coreset.

These dropout layers are active during training for every strategy (not just BALD), so the comparison is fair — all strategies share the same architecture and the same regularisation. They are also active during

FixMatch training (Phase 2), per FIXMATCH_NOTES §9.7.

---

4. Active learning protocol

4.1 Hyperparameters (shared across all strategies, both phases unless noted)

name	value	meaning
`NUM_CYCLES`	6 (main) / 15 (low-budget) / 8 (fixmatch)	cycle 0 trains on init pool; subsequent cycles train on init + picks
`INIT_BUDGET`	0.05 main / 0.025 low-budget / 0.025 fixmatch	initial labeled pool fraction
`BUDGET_PER_CYCLE`	0.05 main / 0.01 low-budget / 0.01 fixmatch	picks added between cycles
`EPOCHS`	40	training epochs per cycle (fresh model each cycle)
`AL_BASE_SEED`	42	`cycle_seed = base_seed + cycle` is reapplied at the top of every cycle
optimizer	`AdamW(lr=1e-3, weight_decay=0.01)`	reset per cycle
loss	`smp.losses.FocalLoss(mode='multiclass', alpha=None, gamma=2.0)`	
scheduler	`CosineAnnealingWarmRestarts(T_0=6*len(loader), T_mult=2, eta_min=1e-6)`	stepped per batch
`BATCH_SIZE`	**4 (Phase 1) / 8 (Phase 2 main) / 4 (Phase 2 FixMatch)**	see §9.6
`INIT_POOL_SEED`	96	shared across strategies and runs (no init pool drift in this report)

Main runs: final labeled pool = 6 × 198 = **1188 images (30% of 3960)**. Low-budget runs: 99 + 14·39 = **645 images (16.3%)**. FixMatch runs: 99 + 7·39 = **372 images (9.4%)**.

4.2 Per-cycle flow

For each cycle `c = 0..N-1`:

1. `set_seed(base_seed + c)` — fully reseeds Python / NumPy / torch CPU and CUDA / cuDNN flags. Same `init_labeled` shared across strategies.
2. Build a `SubsetRandomSampler` DataLoader over the current labeled pool with a per-cycle `torch.Generator(manual_seed=cycle_seed)`.
3. Instantiate a fresh `DeepLabV3Plus`, fresh `AdamW`, fresh scheduler.

4. Train for `EPOCHS=40`.
5. Validate on the 989-image val set; log IoU / F1 / clDice / Loss.
6. If `val_iou` improves over the per-strategy best so far, save `{model_name}_cycle_{c}_best.pth`.
7. If not the last cycle: run the strategy's scoring on the unlabeled pool, take top-K, move those indices from unlabeled → labeled.
8. Log a JPEG composite of the picks (image | GT | prediction) to `result_images/{date}/{strategy}/` and (where the server permits) to ClearML.

After the AL loop, the best checkpoint across all cycles is reloaded as the strategy's final model.

4.3 Strategies tested

Phase 1 has six strategies + fulldata. Phase 2 adds three more (marked **NEW**). All are implemented around the same `run_active_learning(strategy=…)` dispatch so the only thing that differs is the per-image scoring at the end of each cycle.

strategy	scoring	aggregation	notes
`random`	uniform random over unlabeled	—	baseline
`coreset`	k-center greedy on L2-normalized **mean-pooled deepest encoder features** (no MC dropout)	—	Sener & Savarese 2018
`entropy`	per-pixel Shannon entropy of softmax	per-image mean	highest entropy = most uncertain
`margin`	per-pixel `1 − (top1_prob − top2_prob)`	per-image mean	
`bvsb`	per-pixel `top2_prob / top1_prob` (Best vs Second-Best)	per-image mean	
`bald`	mutual information `H(E[p]) − E[H(p)]` via MC-Dropout, `n_mc_samples = 8`	per-image mean	epistemic uncertainty; only stochastic layers (Dropout/Dropout2d) run in train mode, BN stays eval
`power_bvsb` **NEW (Phase 2)**	stochastic top-K via Gumbel-top-K trick on BvSB scores, `β = 2.0`	per-image	$\beta \to \infty \equiv$ vanilla BvSB, $\beta \to 0 \equiv$ random — adds diversity through sampling noise
`bvsb_coreset` **NEW (Phase 2)**	two-stage hybrid: filter top-(3·K) by BvSB, then k-center greedy on encoder embeddings of survivors	per-image	"poor man's BADGE": uncertainty × diversity
`badge` **NEW (Phase 2)**	gradient embedding `(p − one_hot(argmax)) ⊗ decoder_feature`, mean-pooled over the decoder output (Hp × Wp), flattened to `(B, n_classes * D)`; k-means++ seeding selects the batch	per-image grad embedding	Ash et al. 2020 — canonical "uncertainty × diversity" benchmark

For multi-class problems entropy / margin / BvSB differ per pixel, but in **binary segmentation** they all become monotone functions of `|p − 0.5|`. All three aggregate identically (per-image mean over pixels); the per-image rankings differ because each applies a different monotone transform to the per-pixel

probability *before* the mean. This is what makes BvSB (whose transform decays fastest near `p = 1`) suppress confident-background noise the hardest, similar to BALD.

4.4 Full-data baseline (upper bound)

A separate `fulldata` run trains the same model on **all** 3960 labeled samples for 40 epochs and is evaluated on the same val/test sets.

4.5 NEW (Phase 2): FixMatch SSL adaptation

`fixmatch/nb_low_budget_fixmatch.ipynb` keeps the entire AL pipeline (`BvSB` scoring, validation, checkpoint saving) but replaces the supervised training loop with FixMatch. See `fixmatch/FIXMATCH_NOTES.md` for the full design notes.

Per training step:

1. **Labeled forward (grad).** Standard focal loss on the labeled batch.
2. **Weak forward on unlabeled (no_grad).** Geometry-only augmentation (Resize + flips). `argmax(softmax(logits))` is the pseudo-label. `conf_mask = (max_prob ≥ τ)` with **τ = 0.95**.
3. **Strong forward on unlabeled (grad).** Same geometry + photometric distortion + `CoarseDropout`. Cross-entropy against pseudo-label, masked to `conf_mask`. The mean is over **all** pixels (so `L_unsup` is small early when `conf_mask` is sparse and grows with confidence).

Total loss per step: `L = L_sup + λ_u · L_unsup` with **λ_u = 1.0**.

Other knobs:

- `BATCH_SIZE` = 4 labeled, `FIXMATCH_BATCH_SIZE_UNLABELED` = 4 (**μ = 1.0**). Sized so the extra unlabeled forward passes at 1024^2 fit the shared 3090. **Note:** this reverts to the Phase 1 labeled batch size — the FixMatch notebook does *not* inherit Phase 2's BATCH_SIZE=8, so every FixMatch-vs-Phase-2-supervised comparison carries a batch-size confound (§9.6).
- The same MC-Dropout layers run in train mode → small extra regularisation during FixMatch. Same architecture as Phase 2 supervised, so the comparison is fair.
- AL scoring is **unchanged** — BvSB on the un-augmented `train_eval_dataset`, exactly as the supervised pipeline. The only difference is the training objective.

---

5. Logging

- **TensorBoard:** scalars under `logs/{RUN_ID}/`. Per-cycle val metrics are tagged `IoU_{strategy}`, `F1_{strategy}`, `clDice_{strategy}`, `Loss_{strategy}` (N steps each). Per-cycle pool composition is `real_picks_{strategy}` / `old_picks_{strategy}` / `trainSize_{strategy}`. Holdout test metrics are `Test_{IoU,F1,clDice,Loss}/{strategy}`. *Caveat:* training loss is logged as `Loss/{cycle}/train` with **no strategy in the tag** — all strategies overwrite into the same tag, so per-strategy training loss is unrecoverable from TB. FixMatch adds `Loss/{cycle}/train_sup`, `Loss/{cycle}/train_unsup`, `Loss/{cycle}/train_total`, and the diagnostic `FixMatch/{cycle}/mask_frac`.
- **ClearML:** TB scalars are mirrored (one known gap: the `Test_*/fulldata` series is absent from the Phase 2 ClearML tasks despite existing in TB). In Phase 1 the JPEG composites of per-cycle picks and

final `.pkl` artifacts also uploaded successfully. In Phase 2 the internal ClearML files server returns **HTTP 405 on PUT/POST** (verified empirically — see §9.7), so debug-sample JPEGs and artifact binaries silently fail to upload. Phase 2's `run_all.sh` archives all `.pkl` + `.pth` files locally instead.

- **Local pkls:** `{strategy}_results.pkl` files store `{metrics, idxs}` per strategy; `test_results.pkl` stores the held-out test results. Phase 2 fixmatch writes `bvsb_fixmatch_results_{RUN_ID}.pkl` and `test_results_fixmatch_{RUN_ID}.pkl`.

---

6. Runs included in this analysis

6.1 Phase 1 — local runs (launched May 14 – May 26 2026; last run finished May 28)

ClearML task	TB log dir	what was run	n
`al_run_20260514_155205`	`logs/20260514_155205`	full 6-strategy + fulldata	1
`al_run_20260517_005354`	`logs/20260517_005354`	full 6-strategy + fulldata	1
`al_run_20260518_210840`	`logs/20260518_210840`	full 6-strategy + fulldata	1
`al_run_20260521_180410`	`logs/20260521_180410`	**ablation:** coreset on old-only pool (seed 1)	1
`al_run_20260522_190442`	`logs/20260522_190442`	**ablation:** fulldata on old-only pool	1
`al_run_20260523_030634`	`logs/20260523_030634`	**ablation:** coreset on old-only pool (seed 2)	1
`al_run_20260523_134454`	`logs/20260523_134454`	**ablation:** coreset on old-only pool (seed 3)	1
`al_run_20260524_002008`	`logs/20260524_002008`	**regime:** low-budget BvSB + random, seed 1	1
`al_run_20260525_153057`	`logs/20260525_153057`	**regime:** low-budget BvSB + random, seed 2	1
`al_run_20260526_211123`	`logs/20260526_211123`	**regime:** low-budget BvSB + random, seed 3	1

Phase 1 totals (also reflected in `reproducibility_table.txt`):

- Six main strategies (`coreset` / `random` / `entropy` / `margin` / `bvsb` / `bald`): **n=3** seeds-of-noise each.
- `coreset_only_old_data` ablation: **n=3**.
- `fulldata_old_only` ablation: **n=1**.
- Low-budget BvSB / random experiment: **n=3**.

Each main Phase 1 run took ~2 days on a single GPU; the four follow-up notebooks ran sequentially via `jupyter nbconvert --execute --inplace` in a tmux session, spanning **May 22 19:04 → May 25 05:41 ≈ 58.5 hours of wall clock**. Per-notebook timings from pkl mtimes:

- `nb_clean_fulldata`: ~8 h (May 22 19:04 → May 23 03:06)
- `nb_coreset_old_s2`: ~10.6 h (May 23 03:06 → May 23 13:44)
- `nb_coreset_old_s3`: ~10.6 h (May 23 13:44 → May 24 00:19)
- `nb_low_budget`: ~29 h (May 24 00:19 → May 25 05:41)

6.2 Phase 2 — server sweep (May 27 – Jun 10 2026)

Target server: `ssh ultralytics1@192.168.90.199 -p 606` (set up locally as the SSH alias `Ubuntu-Devbox`). Single RTX 3090, shared with neighbors but sufficient headroom for BATCH_SIZE=8 at 1024^2. Project lives at `/workspace/ultralytics1/SyzdykovN/active_learning`. Driver script: `run_all.sh` (on the server at `/workspace/ultralytics1/SyzdykovN/active_learning/run_all.sh`).

```
# what run_all.sh did, conceptually:
N_MAIN=10, N_FM=5
for i in 1..N_MAIN:  jupyter nbconvert --execute --inplace train_clearml.ipynb
for i in 1..N_FM:    (cd fixmatch && nbconvert --execute --inplace nb_low_budget_fixmatch.ipynb)
# between each run: archive_run() moves *.pkl + *.pth into runs/{label}_{RUN_ID}/
```

Wall clock per master.log: launched **2026-05-27 17:19 MSK**, finished **2026-06-10 05:42 MSK — 13 days 12 hours**, every completed run exited 0. (master.log records one false start: the very first launch was killed ~31 s in and relaunched at 17:19:35 — 16 START lines, 15 END lines, all `exit=0`.) Per-run timing from master.log:

segment	per-run	× count	total
`train_clearml.ipynb` (9 strategies + fulldata)	29.9–35.3 h (35.3 h first run, 32.7 h second, ~30 h thereafter)	10	~12.8 days
`nb_low_budget_fixmatch.ipynb` (bvsb only, 8 cycles)	~3.4 h	5	~16.8 h
inter-run sleep (`sleep 30`)	30 s	14	~7 min

Run IDs as archived in `runs/` (sorted chronologically):

```
main_20260527_171943   main_20260529_043947   main_20260530_132006
main_20260531_191455   main_20260602_011121   main_20260603_070712
main_20260604_130218   main_20260605_185740   main_20260607_005438
main_20260608_065232
fixmatch_20260609_125215  fixmatch_20260609_161448  fixmatch_20260609_193633
fixmatch_20260609_225837  fixmatch_20260610_022037
```

Each `main_<RUN_ID>/` archive contains 11 pkls (`{coreset,random,entropy,margin,bvsb,bald,power_bvsb,bvsb_coreset,badge,fulldata}_results.pkl` plus `test_results.pkl`) and 24–32 `*_best.pth` checkpoints (one per "best val IoU" cycle per strategy). Each `fixmatch_<RUN_ID>/` archive contains `bvsb_fixmatch_results_<RUN_ID>.pkl`, `test_results_fixmatch_<RUN_ID>.pkl`, and 5–6 checkpoints. ~25 GB total.

The same archives were copied to `/media/samsepi/data/server_runs/` for local plotting via `plot_server_runs.py` — see §10.

---

7. Headline results

7.1 Phase 1 — main holdout test, mean ± std (n=3 unless noted)

From `reproducibility_table.txt` in this directory.

strategy	n	IoU	F1	clDice	Loss
bald	3	**0.8415 ± 0.0079**	0.9123 ± 0.0047	0.9535 ± 0.0029	0.0195 ± 0.0027
bvsb	3	**0.8391 ± 0.0045**	0.9109 ± 0.0026	0.9520 ± 0.0029	0.0197 ± 0.0021
fulldata (mixed pool, 3960)	3	0.8386 ± 0.0046	0.9095 ± 0.0024	0.9467 ± 0.0042	0.0231 ± 0.0002
margin	3	0.8333 ± 0.0031	0.9069 ± 0.0016	0.9476 ± 0.0021	0.0204 ± 0.0011
fulldata_old_only (3321 old, ablation)	1	0.8323	0.9047	0.9360	0.0258
entropy	3	0.8210 ± 0.0088	0.8996 ± 0.0054	0.9402 ± 0.0106	0.0226 ± 0.0033
coreset_only_old_data (ablation, n=3)	3	0.8193 ± 0.0176	0.8980 ± 0.0117	0.9416 ± 0.0113	0.0266 ± 0.0031
random	3	0.8171 ± 0.0055	0.8973 ± 0.0034	0.9432 ± 0.0014	0.0263 ± 0.0008
coreset	3	0.8080 ± 0.0097	0.8915 ± 0.0058	0.9360 ± 0.0038	0.0249 ± 0.0020

7.2 Phase 1 — low-budget regime (separate experiment, n=3 seeds)

`al_run_20260524_002008` + `al_run_20260525_153057` + `al_run_20260526_211123`. init = 99 (2.5%) + 14 × 39 picks (1%/cycle) → final pool **645 labels (16.3%)**.

strategy	labels	IoU	F1	clDice	Loss
bvsb (low-budget)	645 (~16%)	**0.8332 ± 0.0052**	0.9066 ± 0.0035	0.9475 ± 0.0043	0.0192 ± 0.0006
random (low-budget)	645 (~16%)	0.7926 ± 0.0012	0.8815 ± 0.0011	0.9298 ± 0.0067	0.0257 ± 0.0005

Per-seed TEST IoU (so you can spot the lucky / unlucky seed):

run	bvsb IoU	random IoU	Δ (bvsb − random)
s1 = `20260524_002008`	0.8404	0.7910	+0.0494
s2 = `20260525_153057`	0.8308	0.7929	+0.0379
s3 = `20260526_211123`	0.8284	0.7938	+0.0346
n=3 mean ± std	**0.8332 ± 0.0052**	**0.7926 ± 0.0012**	**+0.0406 ± 0.0064**

Per-metric Δ (bvsb − random) across all four metrics, mean ± std over n=3:

metric	Δ (bvsb − random)
IoU	**+0.0406 ± 0.0064**
F1	+0.0251 ± 0.0046
clDice	+0.0177 ± 0.0108
Loss	−0.0065 ± 0.0011 (lower-is-better; bvsb wins)

s1 (the original) is the **lucky seed for BvSB** — its 0.8404 sits ~1.4σ above the n=3 mean. Random is essentially unchanged across seeds (std=0.0012).

7.3 Phase 2 — main holdout test, mean ± std (n=10), BATCH_SIZE=8

From `server_runs/reproducibility_table.txt`.

strategy	n	IoU	F1	clDice	Loss
margin	10	**0.8434 ± 0.0031**	**0.9138 ± 0.0019**	0.9525 ± 0.0021	**0.0179 ± 0.0007**
entropy	10	**0.8420 ± 0.0066**	0.9130 ± 0.0040	0.9543 ± 0.0042	0.0181 ± 0.0009
bvsb	10	**0.8416 ± 0.0048**	0.9128 ± 0.0028	0.9528 ± 0.0030	0.0188 ± 0.0016
bald	10	**0.8398 ± 0.0052**	0.9117 ± 0.0031	**0.9552 ± 0.0027**	0.0189 ± 0.0011
fulldata (mixed pool, 3960)	10	0.8369 ± 0.0060	0.9063 ± 0.0056	0.9477 ± 0.0029	0.0262 ± 0.0020
`power_bvsb` **NEW**	10	0.8324 ± 0.0066	0.9068 ± 0.0049	0.9517 ± 0.0041	0.0225 ± 0.0018
`bvsb_coreset` **NEW**	10	0.8303 ± 0.0069	0.9059 ± 0.0045	0.9528 ± 0.0039	0.0227 ± 0.0027
`badge` **NEW**	10	0.8256 ± 0.0109	0.9032 ± 0.0067	0.9509 ± 0.0066	0.0229 ± 0.0031
random	10	0.8211 ± 0.0067	0.9002 ± 0.0045	0.9462 ± 0.0053	0.0239 ± 0.0006
coreset	10	0.8097 ± 0.0098	0.8934 ± 0.0060	0.9444 ± 0.0054	0.0238 ± 0.0013

7.4 Phase 2 — FixMatch BvSB (n=5), small-budget regime

8 cycles, init = 99 (2.5%) + 7 × 39 (1%/cycle) → final pool **372 labels (9.4%)**. Same model, same BvSB scoring, supervised loop replaced with FixMatch (`τ = 0.95`, `λ_u = 1.0`).

metric	mean ± std	min	max
IoU	**0.8218 ± 0.0045**	0.8146	0.8286
F1	0.8996 ± 0.0031	0.8947	0.9044
clDice	0.9424 ± 0.0044	0.9373	0.9506
Loss	0.0235 ± 0.0008	0.0224	0.0244

Head-to-head (all Phase 2 runs, but **not** batch-matched: FixMatch trains at labeled batch 4, the supervised comparators at batch 8 — see §9.6):

comparator	TEST IoU	Δ (fixmatch − comparator)
FixMatch BvSB @372 (n=5)	0.8218 ± 0.0045	—
Phase 2 main bvsb @1188 (n=10)	0.8416 ± 0.0048	**−0.0198**
Phase 2 fulldata @3960 (n=10)	0.8369 ± 0.0060	**−0.0150**

7.5 Cross-phase observations (Phase 1 vs Phase 2)

Same strategies, same data, different (n, BATCH_SIZE) — the table below makes the shifts explicit:

strategy	Phase 1 IoU (n=3, B=4)	Phase 2 IoU (n=10, B=8)	Δ (Phase 2 − Phase 1)
margin	0.8333 ± 0.0031	**0.8434 ± 0.0031**	**+0.0101**
entropy	0.8210 ± 0.0088	**0.8420 ± 0.0066**	**+0.0210**
bvsb	0.8391 ± 0.0045	0.8416 ± 0.0048	+0.0025
bald	0.8415 ± 0.0079	0.8398 ± 0.0052	−0.0017
fulldata	0.8386 ± 0.0046	0.8369 ± 0.0060	−0.0017
random	0.8171 ± 0.0055	0.8211 ± 0.0067	+0.0039
coreset	0.8080 ± 0.0097	0.8097 ± 0.0098	+0.0017

Three things jump out:

1. `entropy` **and** `margin` **improved markedly in Phase 2** — +2.1pp and +1.0pp respectively. Two candidate explanations, neither verified: (a) BATCH_SIZE=8 might yield better-calibrated softmax probabilities and hence better uncertainty scores — plausible, but we never measured calibration; (b) the §9.8 scoring fix — Phase 1 main runs scored the unlabeled pool through *augmented* images, all Phase 2 runs score un-augmented ones, which directly changes entropy/margin pick quality. The entropy shift is large (~2.4 Phase-1 σ; Welch t ≈ 3.2) but rests on n=3 Phase 1 runs. Treat the shift as real, its cause as unresolved.
2. `fulldata` **drifted slightly down** (−0.17pp) and `bald` **drifted slightly down** (−0.17pp). Both within their own std. With B=8 the batch-norm running statistics evolve differently — small effect.
3. **The ranking shifted at the top.** Phase 1 had `bald ≈ bvsb ≈ fulldata > margin > entropy > random > coreset`. Phase 2 has `margin ≈ entropy ≈ bvsb ≈ bald > fulldata > power_bvsb > bvsb_coreset > badge > random > coreset`. The four uncertainty methods are now within 0.004 IoU of each other (all within their own ± std). Treat the within-top-4 ordering as noise.

7.6 Conclusions that survive both phases

Cross-phase confound reminder. Phase 1 used BATCH_SIZE=4, n=3 seeds, and pre-fix scoring (§9.8); Phase 2 used BATCH_SIZE=8, n=10 seeds, post-fix scoring. Direct numeric comparison shifts absolute IoU per strategy by −0.2pp to +2.1pp (see §7.5 and §9.6); the ranking and Δ-vs-baseline interpretations carry over, point estimates do not. Points 1–3 below hold at *each* phase's batch size independently; points 4–8 are single-phase findings, labeled as such.

1. **All four uncertainty methods (entropy / margin / bvsb / bald) beat random.** In Phase 1 (n=3, B=4) the margin was +0.4 to +2.4pp; in Phase 2 (n=10, B=8) it's +1.9 to +2.2pp. The Phase 2 version is

robust for all four; in Phase 1 the entropy endpoint (+0.4pp at n=3, ~0.5σ) is within noise, so the all-four claim rests mainly on Phase 2.

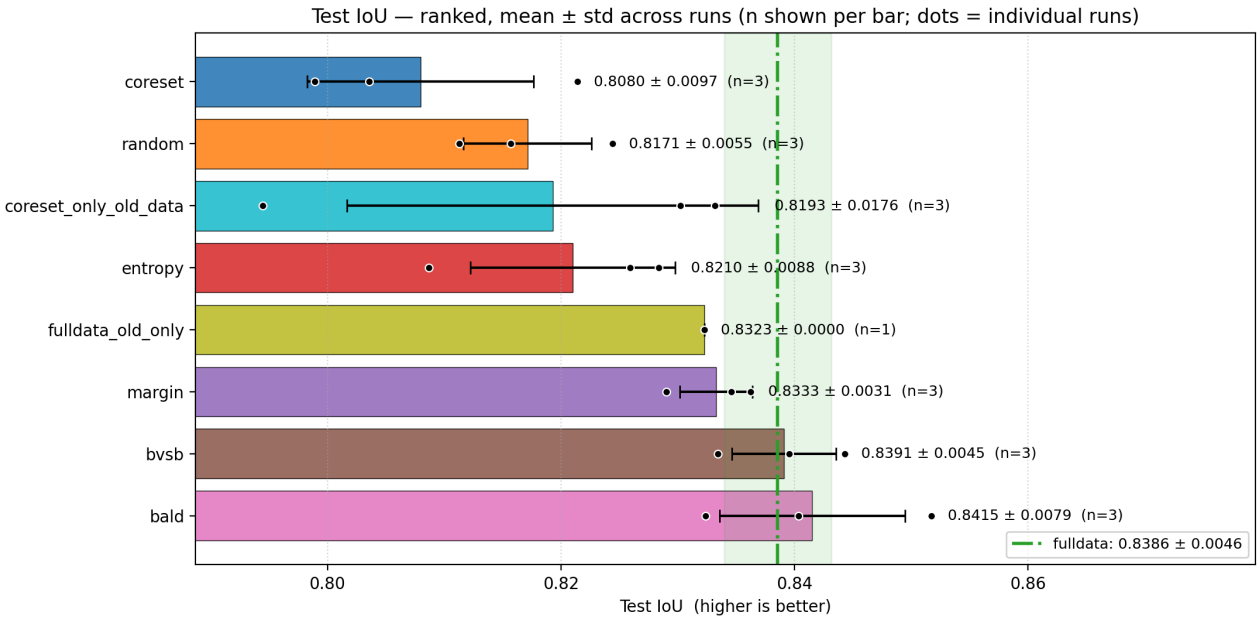
2. **Uncertainty methods match or exceed fulldata at 30 % of the labels.** Phase 1: BALD and BvSB sit within ±1σ of fulldata. Phase 2: margin, entropy, BvSB, and BALD all sit **above** fulldata, with margin exceeding it by ~1.1σ of fulldata (Δ = +0.0066, σ_fulldata = 0.0060; against the std of the difference $\sqrt{(\sigma^2_\text{margin} + \sigma^2_\text{fulldata})} \approx 0.0068$ that is Δ ≈ 0.97σ; Welch t ≈ 2.9). The Phase 2 loss column is more striking: uncertainty methods sit at ~0.018, fulldata at ~0.026 → **~30 % lower test loss** with 70 % fewer labels (**Phase 2 only** — the Phase 1 loss gap is a more modest ~11 %). AL is functioning as data curation that filters out the noisy `real` pool; see §8.4.
3. **Coreset is the only AL strategy that loses to random** at both batch sizes. Phase 1 (n=3, B=4): −0.92pp; Phase 2 (n=10, B=8): −1.14pp. Phase 2 is statistically solid (Welch t = −2.9, p ≈ 0.01); Phase 1 is directionally consistent but not significant on its own (p ≈ 0.3 at n=3). K-center is genuinely a poor criterion for this task; see §8.1 / §8.5.
4. **AL gains concentrate where labels are scarce** (Phase 1 low-budget, n=3, B=4 only — Phase 2 has no low-budget *supervised* run). +2.20pp at 30 %, **+4.06pp ± 0.64pp at 16 %**. Every individual low-budget seed shows Δ ≥ +3.4pp (per-seed +4.94 / +3.79 / +3.46pp). Mittal's "random is hard to beat" applies above some budget; below it, AL clearly helps.
5. **The "noisy pool was the problem and AL routed around it" hypothesis only partially survives.** Two Phase 1 ablations undermine the strong version:
6. `coreset_only_old_data` n=3 gives only +1.1pp over combined-pool coreset, and the std (0.0176) is larger than the effect. Removing the noisy pool helps coreset a little, not a lot. (§8.5)
7. `fulldata_old_only` n=1 is **worse** than mixed-pool fulldata by 0.6pp. So the noisy `real` images aren't pure noise — they add useful supervision when paired with clean labels at full scale. (§8.6)
8. **The three new diversity-aware strategies (power_bvsb, bvsb_coreset, badge) all underperform plain BvSB** (Phase 2 only, n=10). All three sit between random and the uncertainty methods (badge's edge over random is within noise, §8.8). Same mechanism as coreset: pulling picks toward atypical / outlier regions of feature space concentrates training on the noisy `real` pool. (§8.8)
9. **FixMatch SSL did not close the small-budget gap on its own** (Phase 2, n=5). At 9.4 % labels, FixMatch BvSB = 0.8218, vs Phase 2 supervised BvSB @30 % = 0.8416 (Δ = −2.0pp) and fulldata = 0.8369 (Δ = −1.5pp; both comparisons also batch-confounded — FixMatch trains at labeled batch 4, the comparators at 8, §9.6). We cannot conclude "FixMatch hurts" without a matched random+FixMatch baseline. (§8.9)
10. **BvSB at 16 % labels is *near* fulldata but not matching** (Phase 1 low-budget, n=3). 0.8332 ± 0.0052 vs 0.8386 ± 0.0046 — a 0.5pp gap, ~1σ below. The original n=1 0.8404 figure that "matched fulldata at 16 %" was the lucky seed s1.

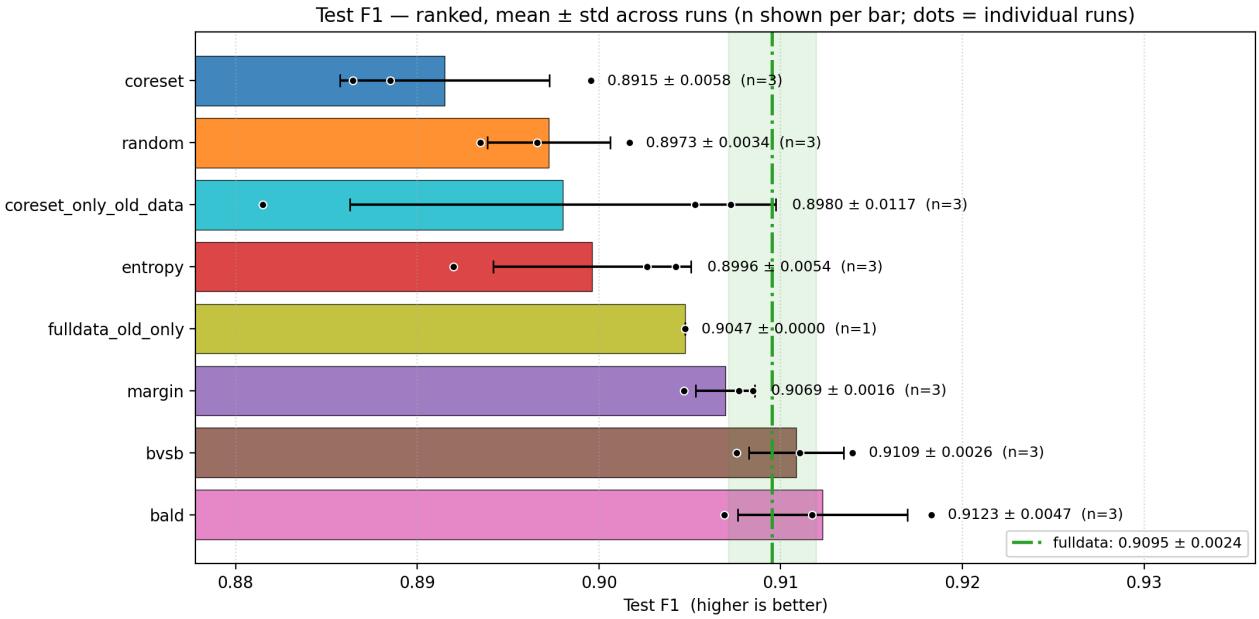
---

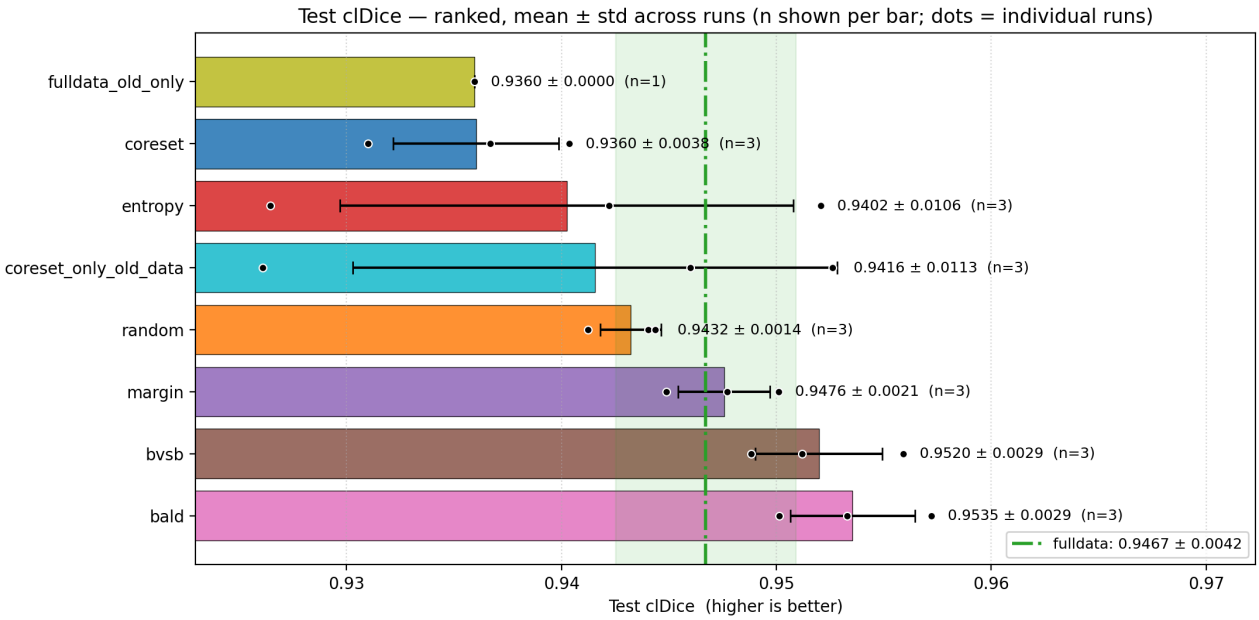
8. Detailed analyses with plots

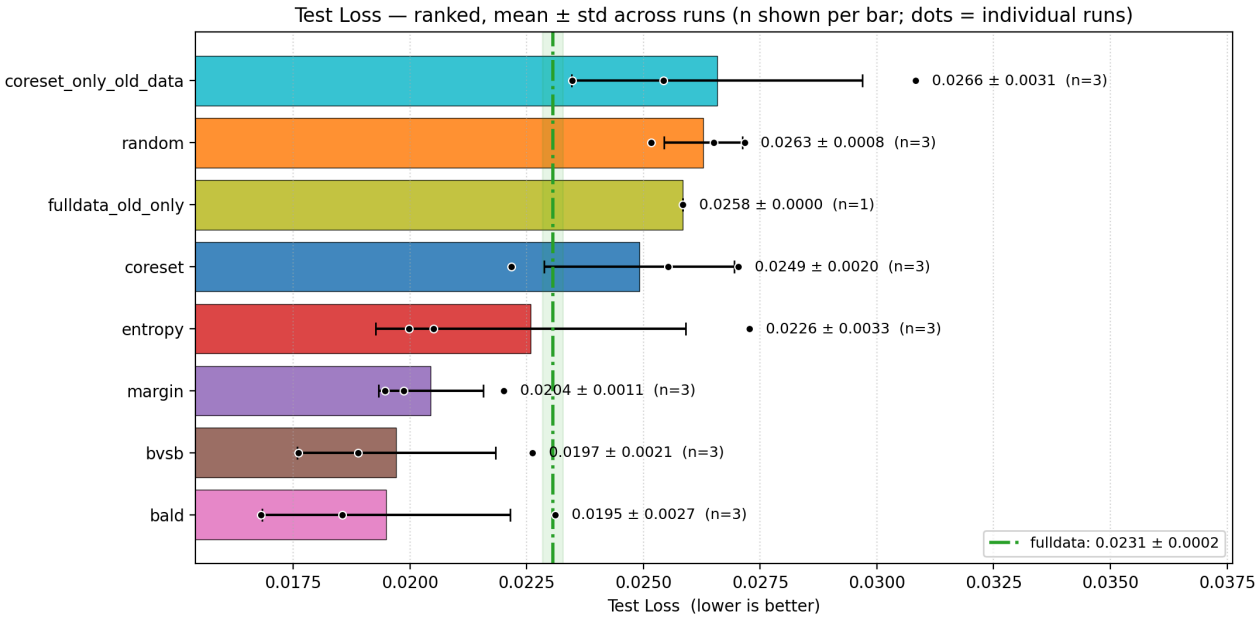
8.1 Test metric ranking — both phases

Phase 1 (n=3, BATCH_SIZE=4) — horizontal ranked bars with individual-run dots and the mixed-pool fulldata reference line. The two ablation rows (`fulldata_old_only` n=1, `coreset_only_old_data` n=3) are included.

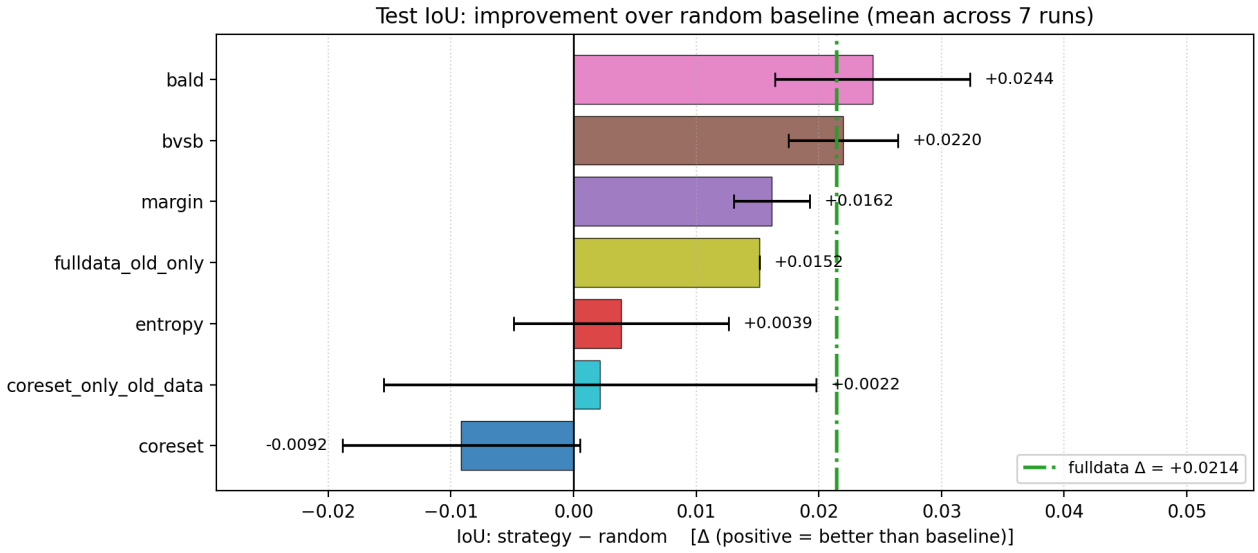


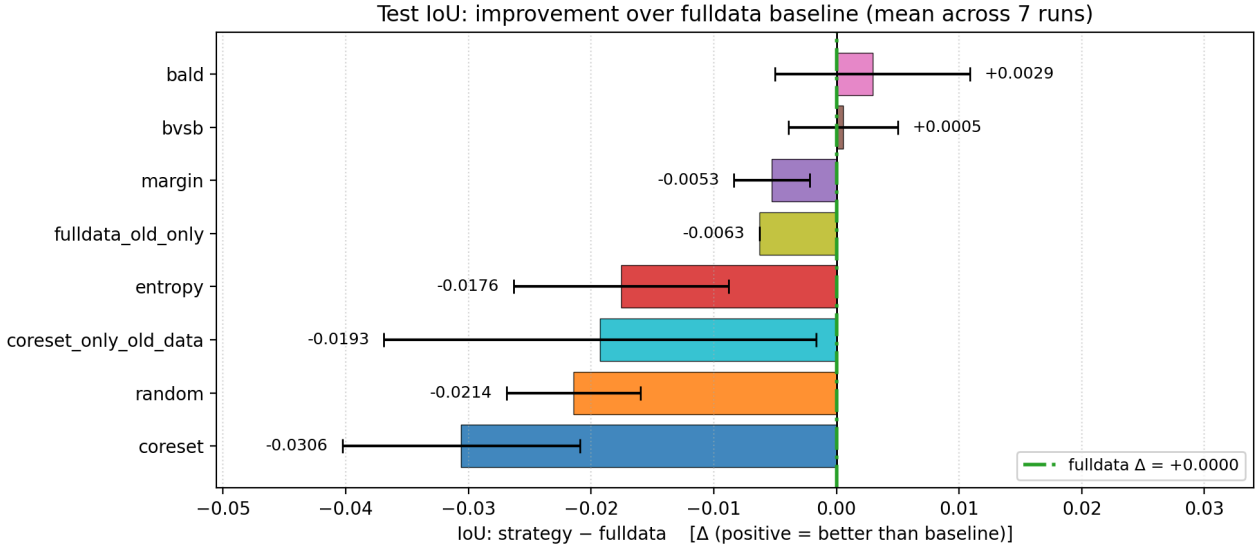


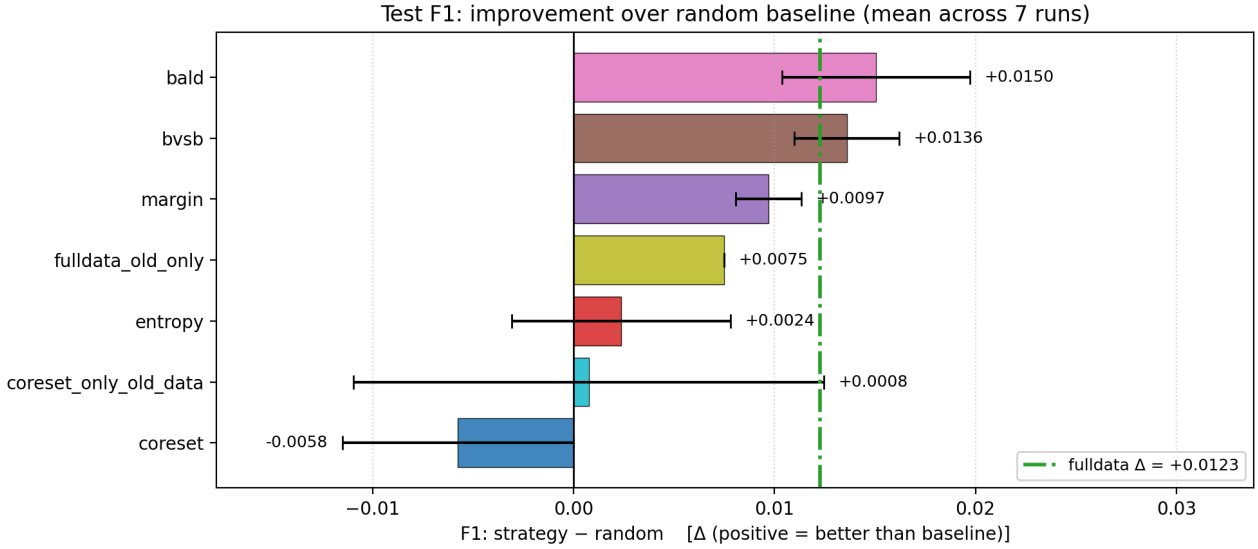


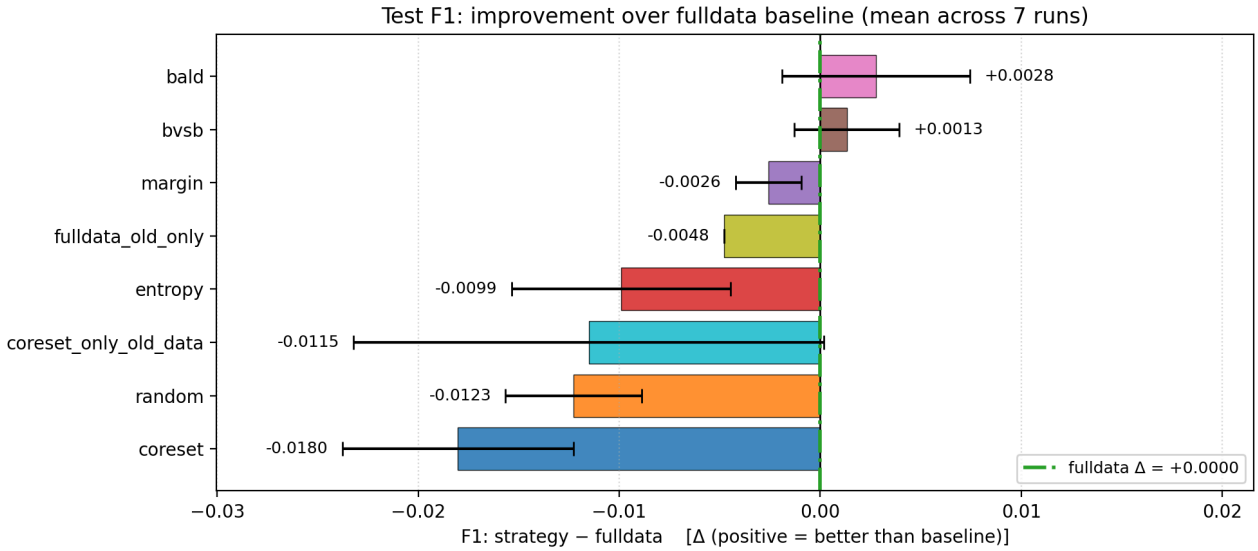


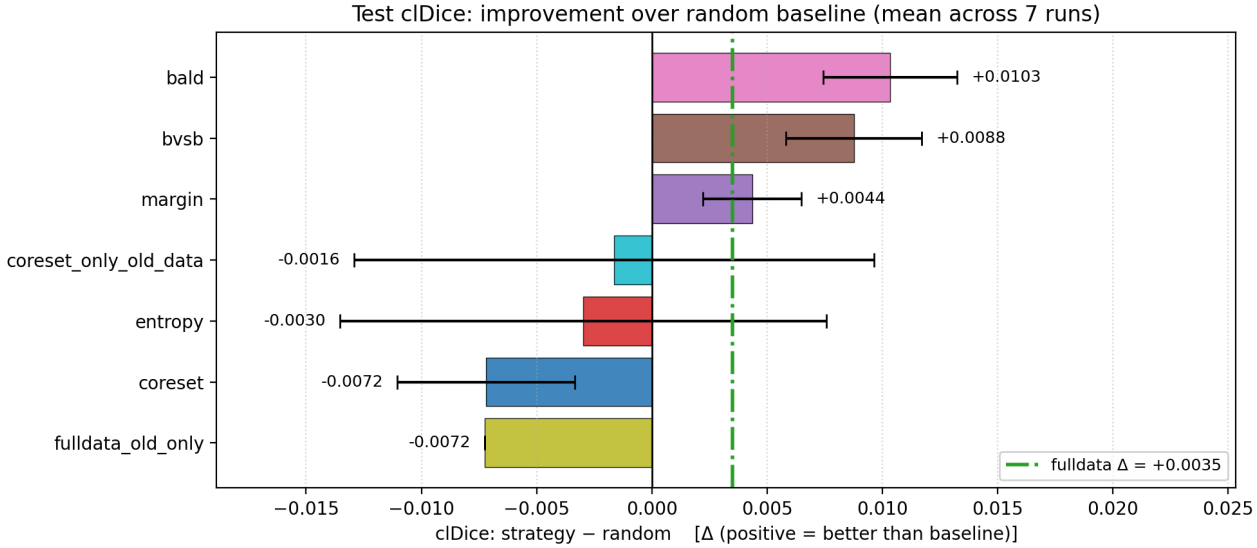
Δ-vs-random and Δ-vs-fulldata views: positive = better than baseline (for Loss the sign is flipped so "+ = better" still means lower loss).

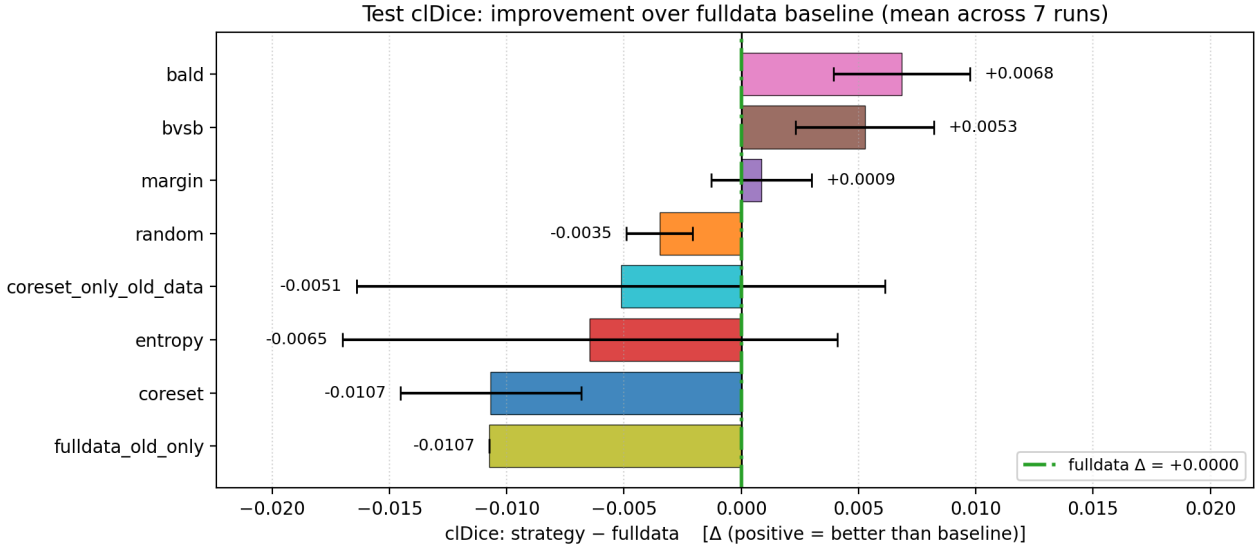

Test IoU: improvement over random baseline (mean across 7 runs)
bald
bvsb
margin
fulldata_old_only
entropy
coreset_only_old_data
coreset
+0.0244
+0.0220
+0.0162
+0.0152
+0.0039
+0.0022
-0.0092
fulldata Δ = +0.0214
−0.02
−0.01
0.00
0.01
0.02
0.03
0.04
0.05
IoU: strategy − random [Δ (positive = better than baseline)]

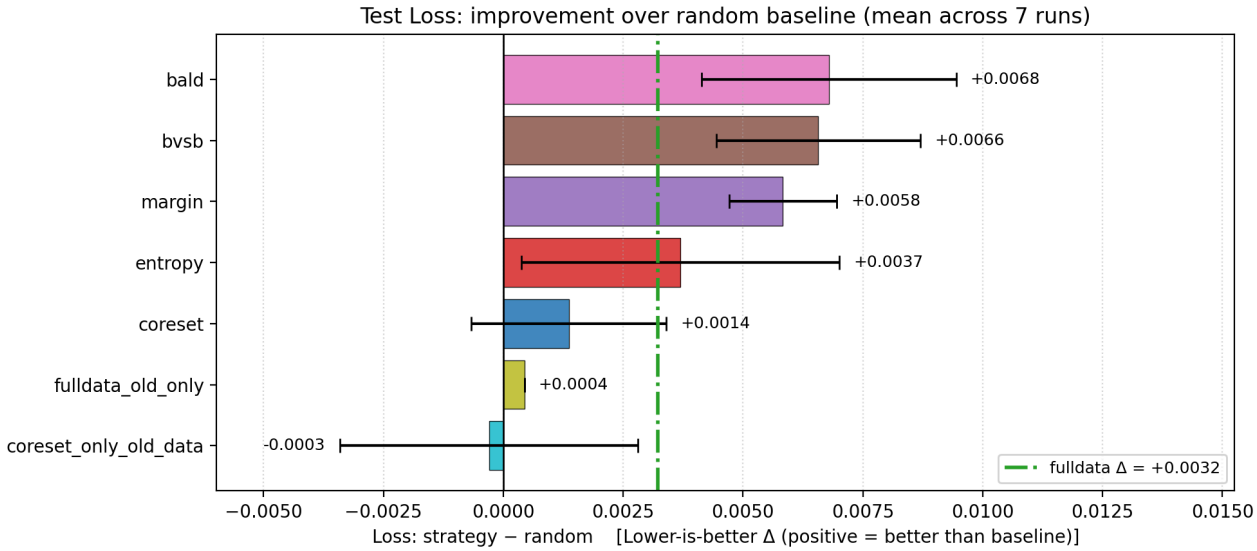

Test IoU: improvement over fulldata baseline (mean across 7 runs)
bald
bvsb
margin
fulldata_old_only
entropy
coreset_only_old_data
random
coreset
+0.0029
+0.0005
-0.0053
-0.0063
-0.0176
-0.0193
-0.0214
-0.0306
fulldata Δ = +0.0000
−0.05
−0.04
−0.03
−0.02
−0.01
0.00
0.01
0.02
0.03
IoU: strategy − fulldata [Δ (positive = better than baseline)]

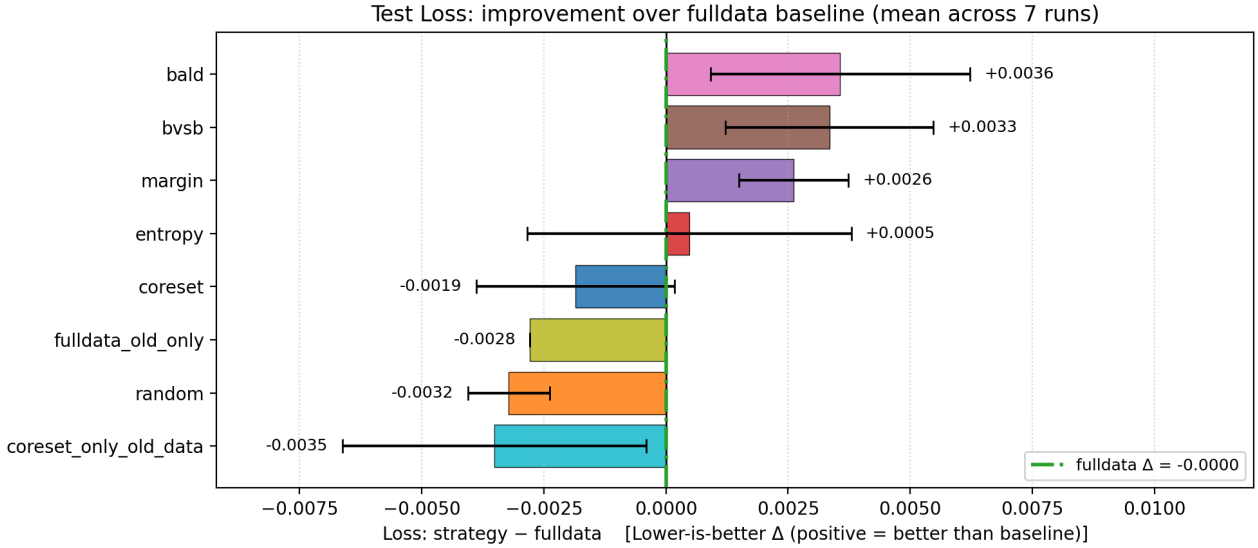

Test F1: improvement over random baseline (mean across 7 runs)
bald
bvsb
margin
fulldata_old_only
entropy
coreset_only_old_data
coreset
+0.0150
+0.0136
+0.0097
+0.0075
+0.0024
+0.0008
-0.0058
fulldata Δ = +0.0123
−0.01
0.00
0.01
0.02
0.03
F1: strategy − random [Δ (positive = better than baseline)]


Test F1: improvement over fulldata baseline (mean across 7 runs)
bald
bvsb
margin
fulldata_old_only
entropy
coreset_only_old_data
random
coreset
+0.0028
+0.0013
-0.0026
-0.0048
-0.0099
-0.0115
-0.0123
-0.0180
fulldata Δ = +0.0000
−0.03
−0.02
−0.01
0.00
0.01
0.02
F1: strategy − fulldata [Δ (positive = better than baseline)]

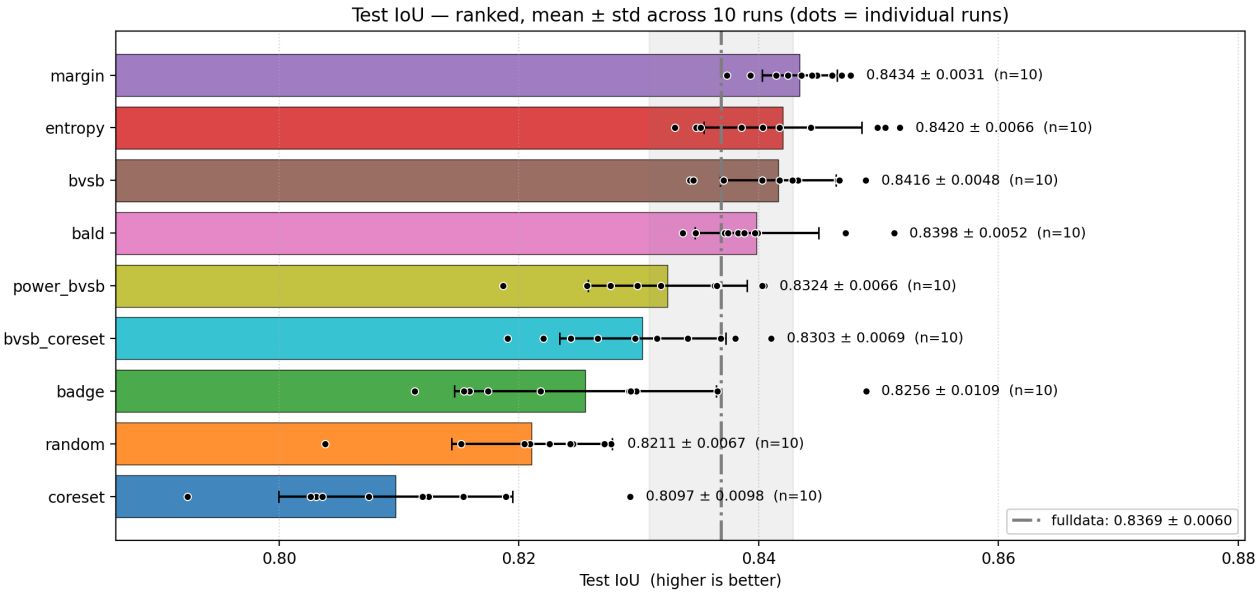

Test clDice: improvement over random baseline (mean across 7 runs)
bald
bvsb
margin
coreset_only_old_data
entropy
coreset
fulldata_old_only
+0.0103
+0.0088
+0.0044
-0.0016
-0.0030
-0.0072
-0.0072
fulldata Δ = +0.0035
−0.015
−0.010
−0.005
0.000
0.005
0.010
0.015
0.020
0.025
clDice: strategy − random [Δ (positive = better than baseline)]

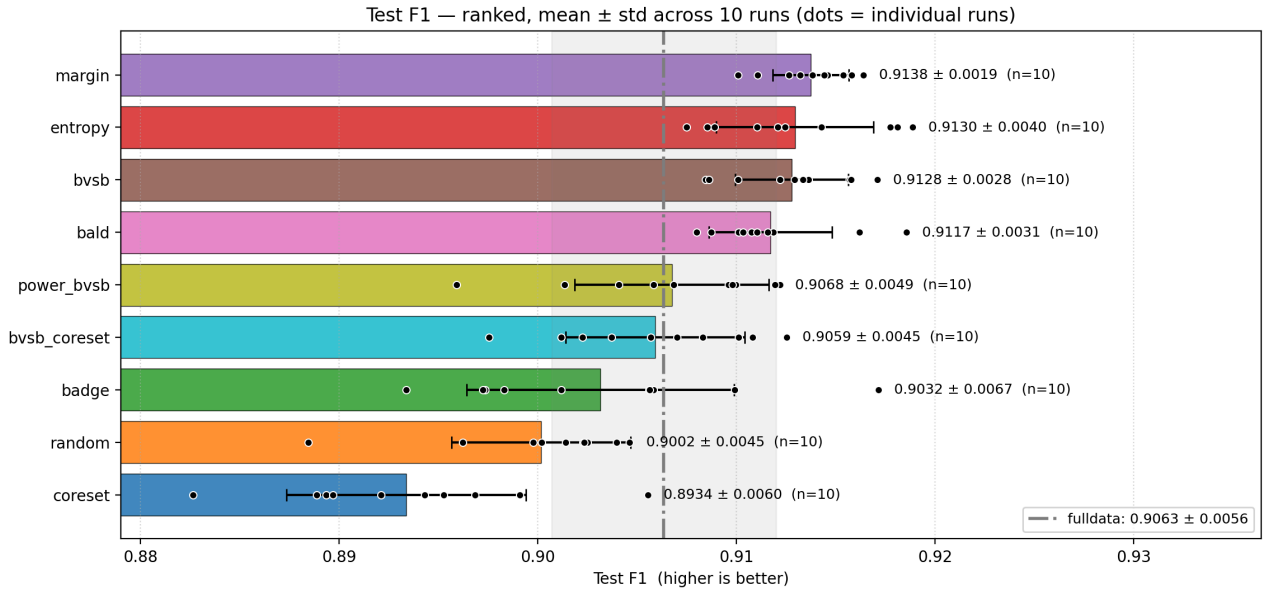

Test clDice: improvement over fulldata baseline (mean across 7 runs)
bald
bvsb
margin
random
coreset_only_old_data
entropy
coreset
fulldata_old_only
+0.0068
+0.0053
+0.0009
-0.0035
-0.0051
-0.0065
-0.0107
-0.0107
fulldata Δ = +0.0000
−0.020
−0.015
−0.010
−0.005
0.000
0.005
0.010
0.015
0.020
clDice: strategy − fulldata [Δ (positive = better than baseline)]

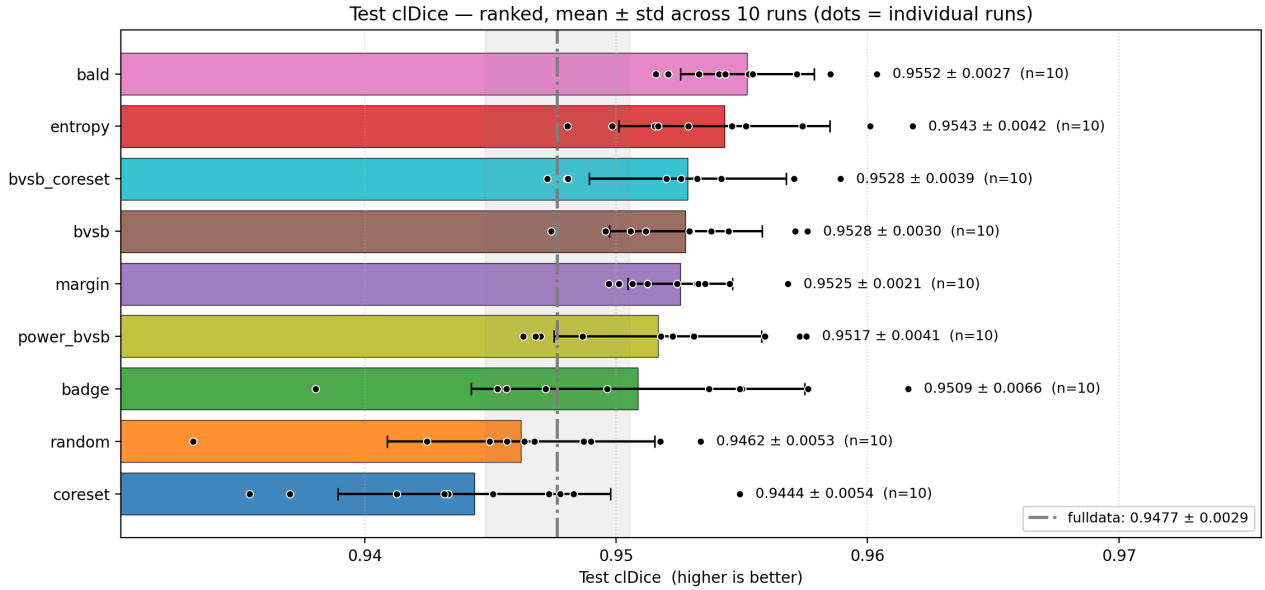


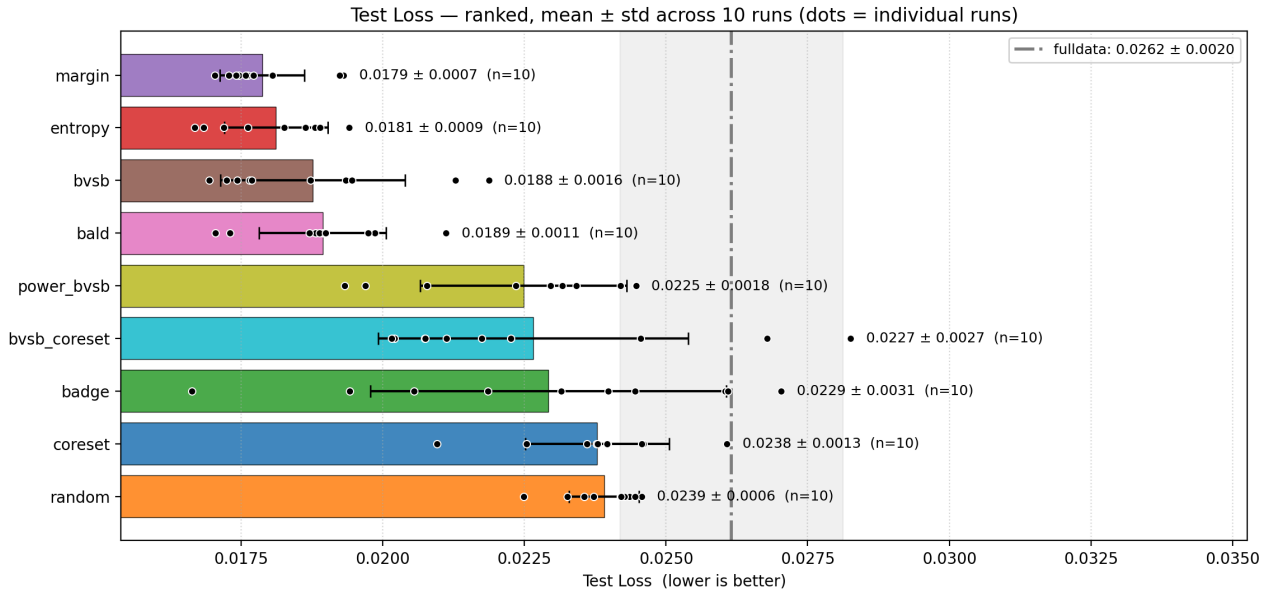


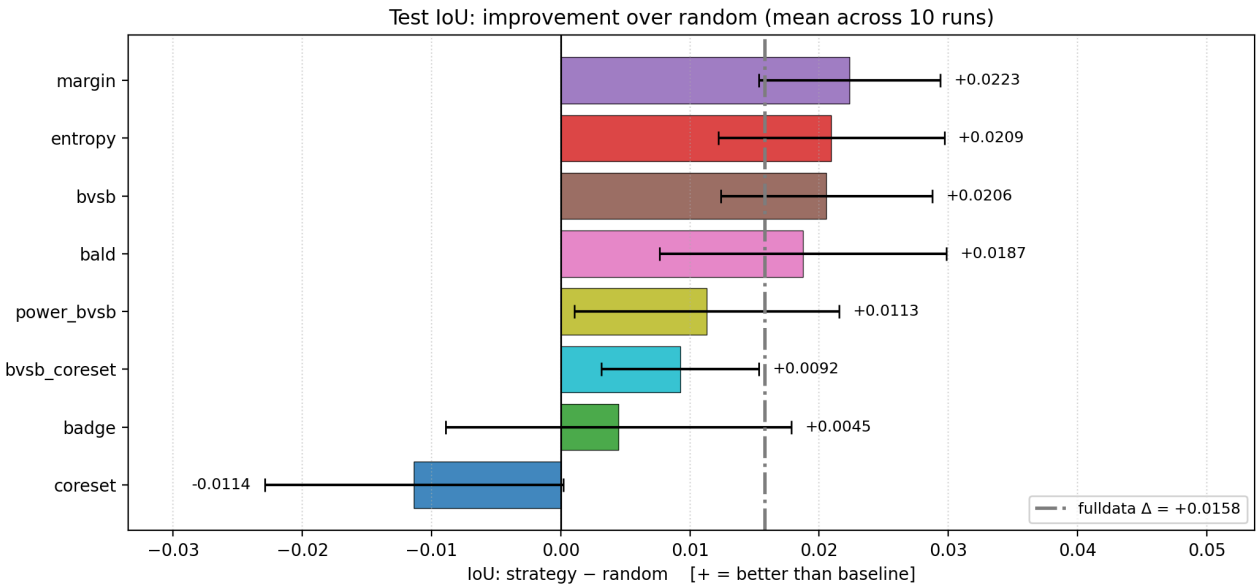
Phase 2 (n=10, BATCH_SIZE=8) — same plots, including the three new strategies. The fulldata reference line drops below margin / entropy / bvsb / bald, visualising the §7.6.2 finding.

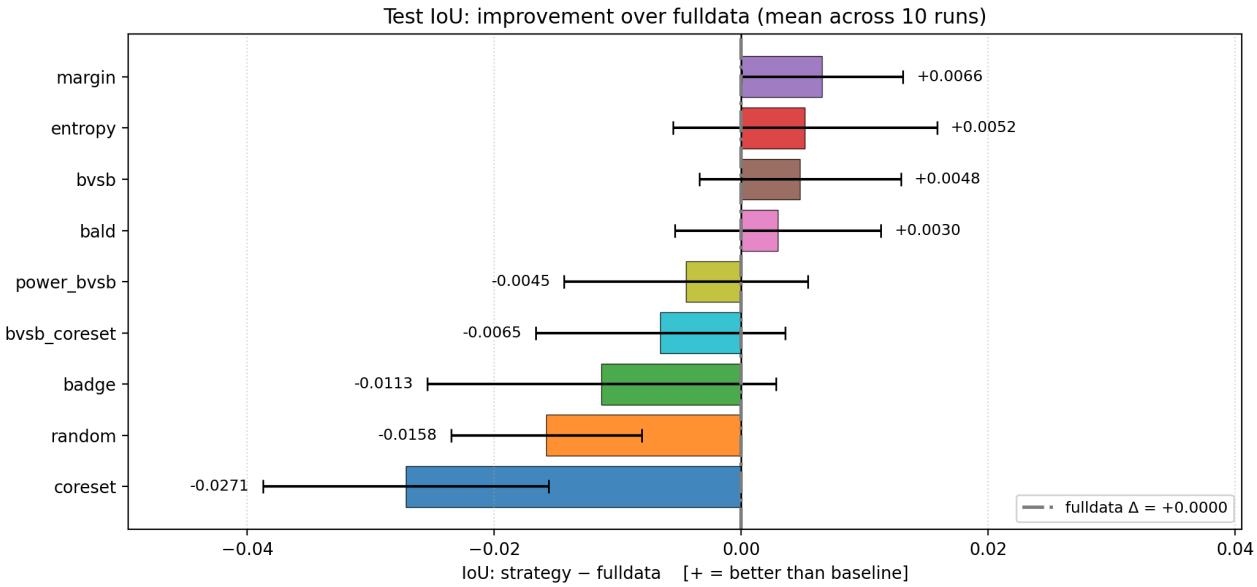


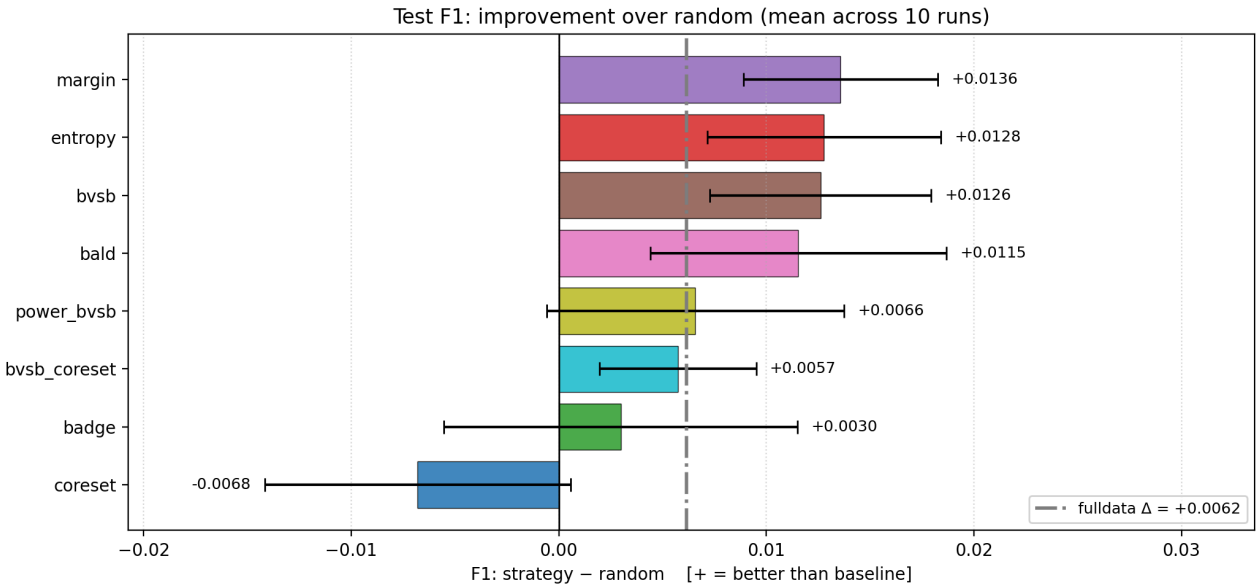

Test F1 — ranked, mean ± std across 10 runs (dots = individual runs)
margin
entropy
bvsb
bald
power_bvsb
bvsb_coreset
badge
random
coreset
0.9138 ± 0.0019 (n=10)
0.9130 ± 0.0040 (n=10)
0.9128 ± 0.0028 (n=10)
0.9117 ± 0.0031 (n=10)
0.9068 ± 0.0049 (n=10)
0.9059 ± 0.0045 (n=10)
0.9032 ± 0.0067 (n=10)
0.9002 ± 0.0045 (n=10)
0.8934 ± 0.0060 (n=10)
fulldata: 0.9063 ± 0.0056
0.88
0.89
0.90
0.91
0.92
0.93
Test F1 (higher is better)

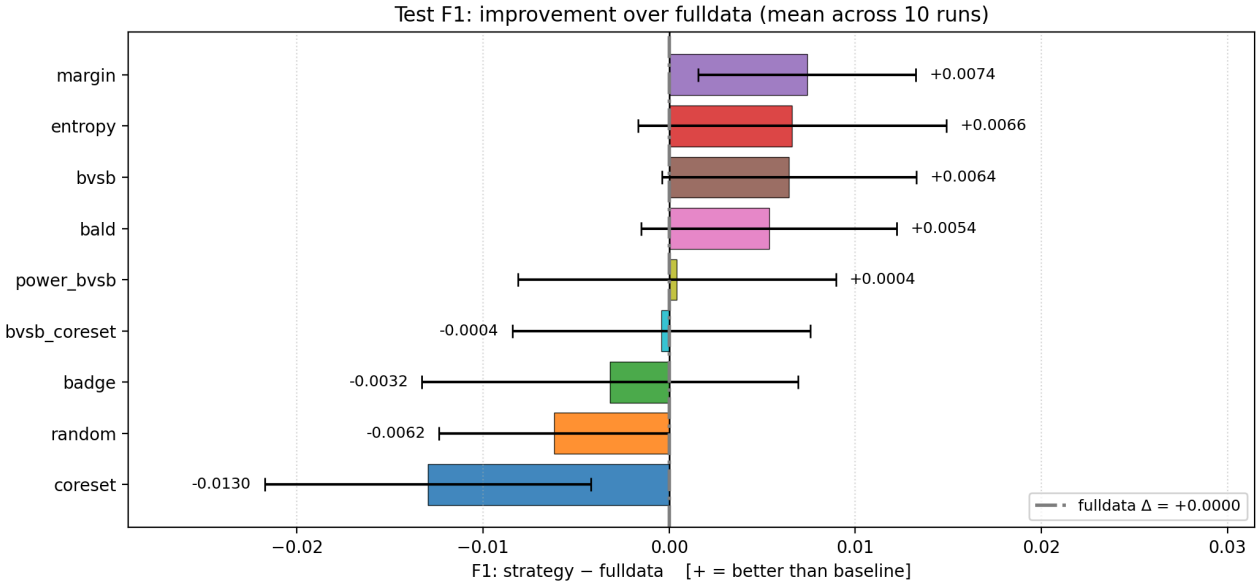

Test clDice — ranked, mean ± std across 10 runs (dots = individual runs)
bald
entropy
bvsb_coreset
bvsb
margin
power_bvsb
badge
random
coreset
0.9552 ± 0.0027 (n=10)
0.9543 ± 0.0042 (n=10)
0.9528 ± 0.0039 (n=10)
0.9528 ± 0.0030 (n=10)
0.9525 ± 0.0021 (n=10)
0.9517 ± 0.0041 (n=10)
0.9509 ± 0.0066 (n=10)
0.9462 ± 0.0053 (n=10)
0.9444 ± 0.0054 (n=10)
fulldata: 0.9477 ± 0.0029
0.94
0.95
0.96
0.97
Test clDice (higher is better)

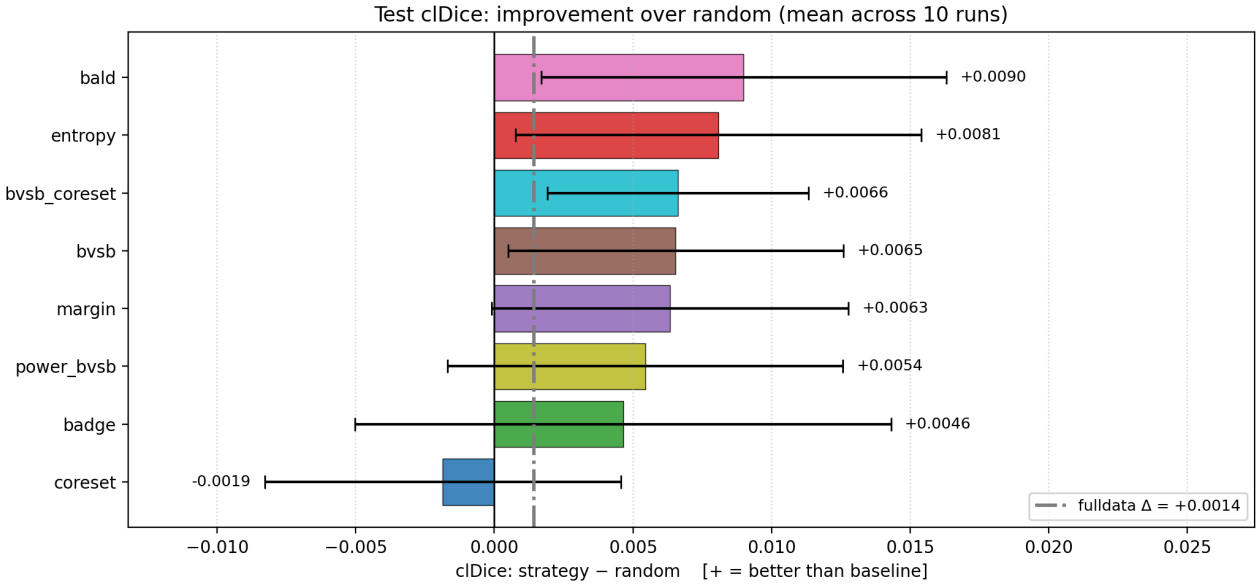

Test Loss — ranked, mean ± std across 10 runs (dots = individual runs)
margin
entropy
bvsb
bald
power_bvsb
bvsb_coreset
badge
coreset
random
0.0179 ± 0.0007 (n=10)
0.0181 ± 0.0009 (n=10)
0.0188 ± 0.0016 (n=10)
0.0189 ± 0.0011 (n=10)
0.0225 ± 0.0018 (n=10)
0.0227 ± 0.0027 (n=10)
0.0229 ± 0.0031 (n=10)
0.0238 ± 0.0013 (n=10)
0.0239 ± 0.0006 (n=10)
fulldata: 0.0262 ± 0.0020
0.0175
0.0200
0.0225
0.0250
0.0275
0.0300
0.0325
0.0350
Test Loss (lower is better)

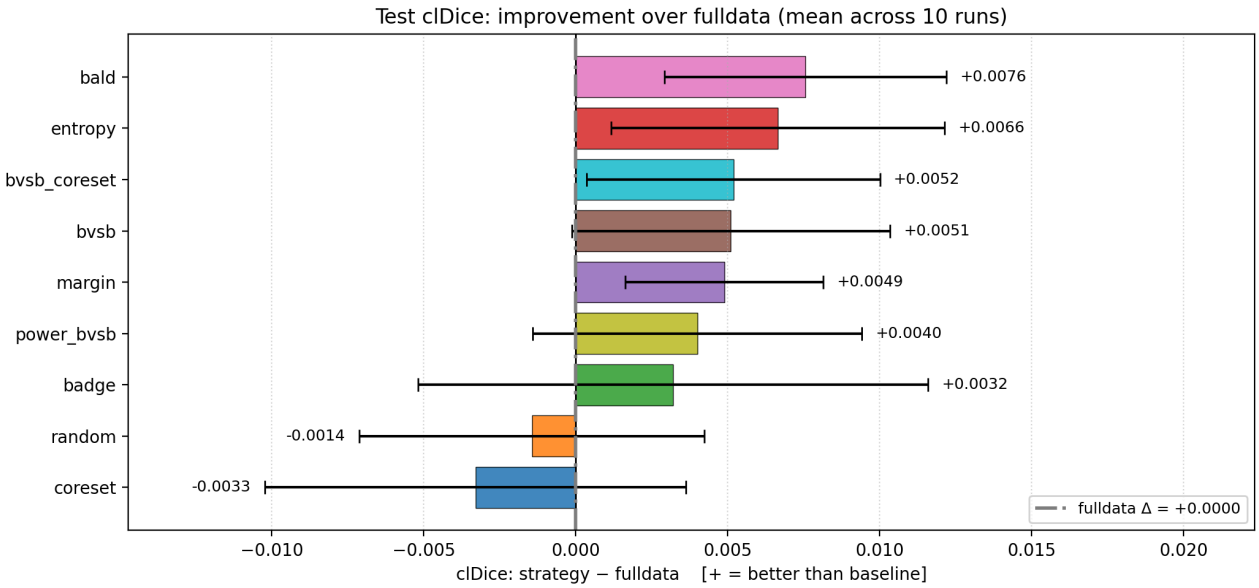

Test IoU: improvement over random (mean across 10 runs)
margin
entropy
bvsb
bald
power_bvsb
bvsb_coreset
badge
coreset
+0.0223
+0.0209
+0.0206
+0.0187
+0.0113
+0.0092
+0.0045
-0.0114
fulldata Δ = +0.0158
−0.03
−0.02
−0.01
0.00
0.01
0.02
0.03
0.04
0.05
IoU: strategy − random [+ = better than baseline]

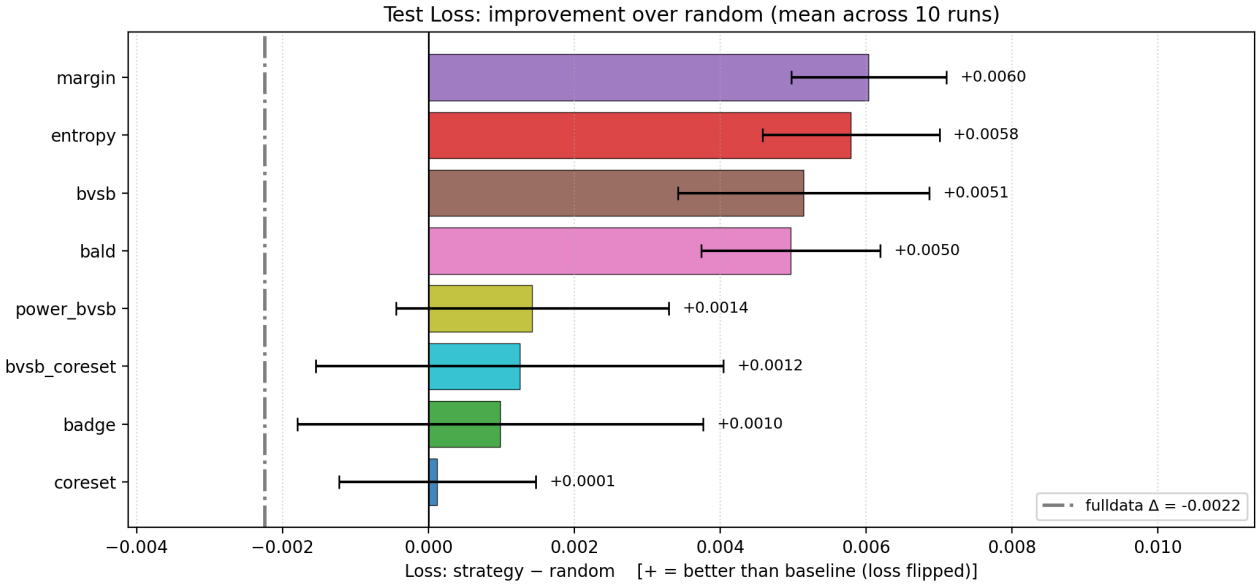

Test IoU: improvement over fulldata (mean across 10 runs)
margin
entropy
bvsb
bald
power_bvsb
bvsb_coreset
badge
random
coreset
+0.0066
+0.0052
+0.0048
+0.0030
-0.0045
-0.0065
-0.0113
-0.0158
-0.0271
fulldata Δ = +0.0000
−0.04
−0.02
0.00
0.02
0.04
IoU: strategy − fulldata [+ = better than baseline]

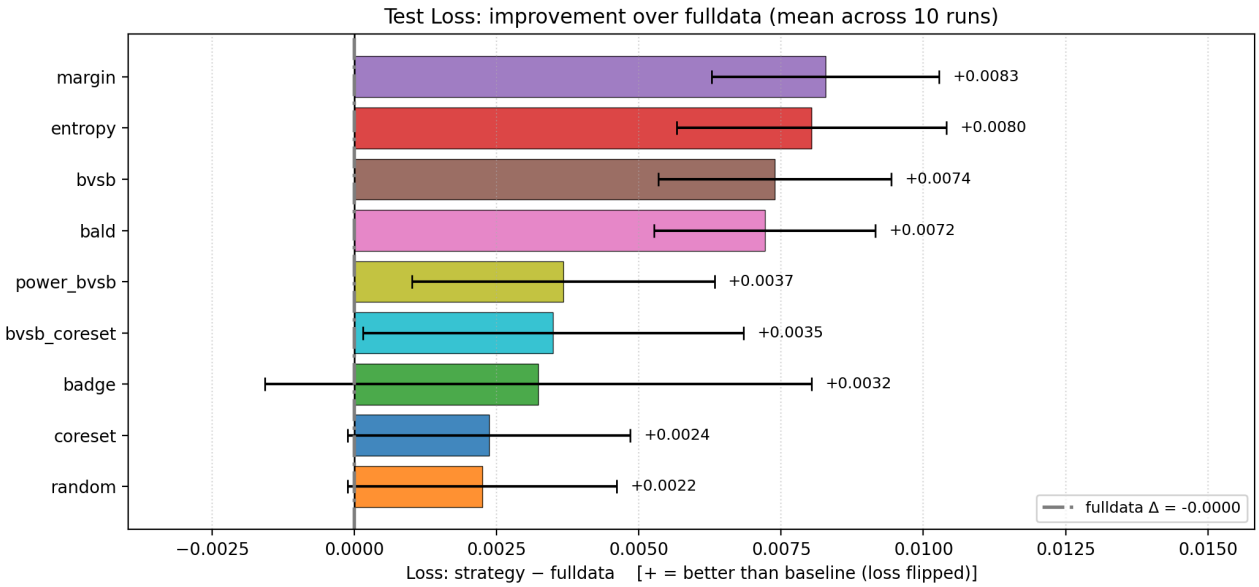

Test F1: improvement over random (mean across 10 runs)
margin
entropy
bvsb
bald
power_bvsb
bvsb_coreset
badge
coreset
+0.0136
+0.0128
+0.0126
+0.0115
+0.0066
+0.0057
+0.0030
-0.0068
fulldata Δ = +0.0062
−0.02
−0.01
0.00
0.01
0.02
0.03
F1: strategy − random [+ = better than baseline]


Test F1: improvement over fulldata (mean across 10 runs)
margin
entropy
bvsb
bald
power_bvsb
bvsb_coreset
badge
random
coreset
+0.0074
+0.0066
+0.0064
+0.0054
+0.0004
-0.0004
-0.0032
-0.0062
-0.0130
fulldata Δ = +0.0000
−0.02
−0.01
0.00
0.01
0.02
0.03
F1: strategy − fulldata [+ = better than baseline]


Test clDice: improvement over random (mean across 10 runs)
bald
entropy
bvsb_coreset
bvsb
margin
power_bvsb
badge
coreset
+0.0090
+0.0081
+0.0066
+0.0065
+0.0063
+0.0054
+0.0046
-0.0019
fulldata Δ = +0.0014
−0.010
−0.005
0.000
0.005
0.010
0.015
0.020
0.025
clDice: strategy − random [+ = better than baseline]


Test clDice: improvement over fulldata (mean across 10 runs)
bald
entropy
bvsb_coreset
bvsb
margin
power_bvsb
badge
random
coreset
+0.0076
+0.0066
+0.0052
+0.0051
+0.0049
+0.0040
+0.0032
-0.0014
-0.0033
fulldata Δ = +0.0000
−0.010
−0.005
0.000
0.005
0.010
0.015
0.020
clDice: strategy − fulldata [+ = better than baseline]

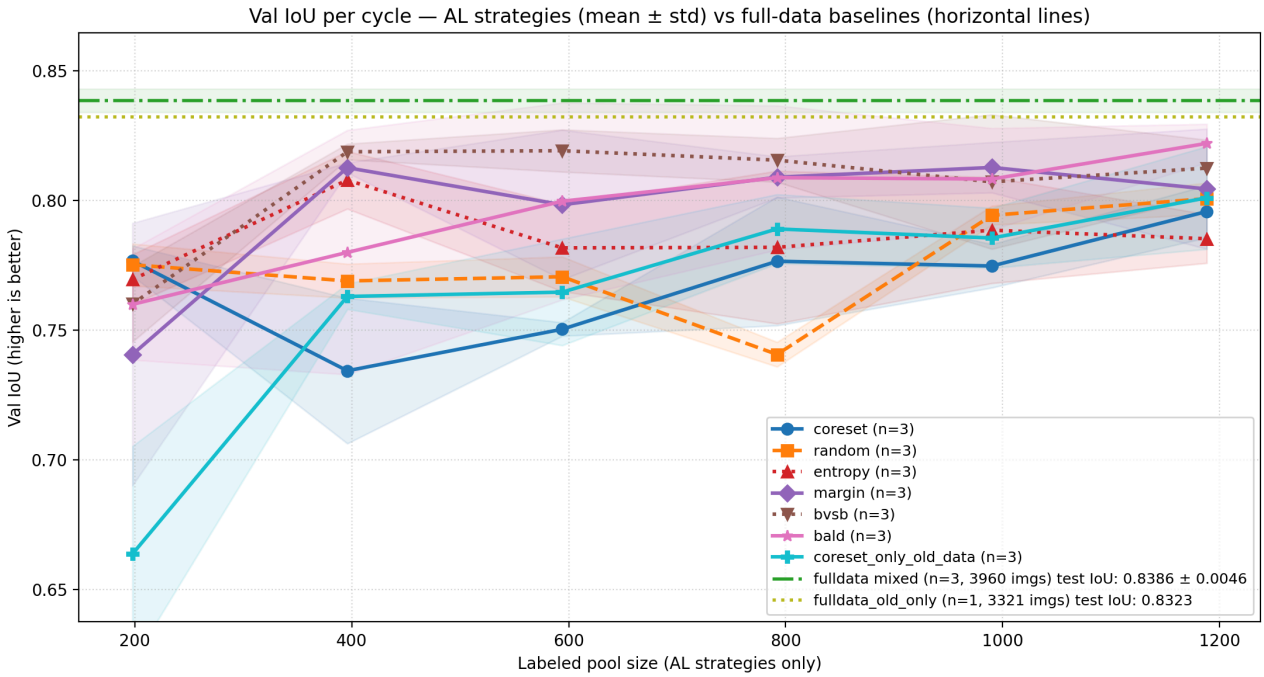


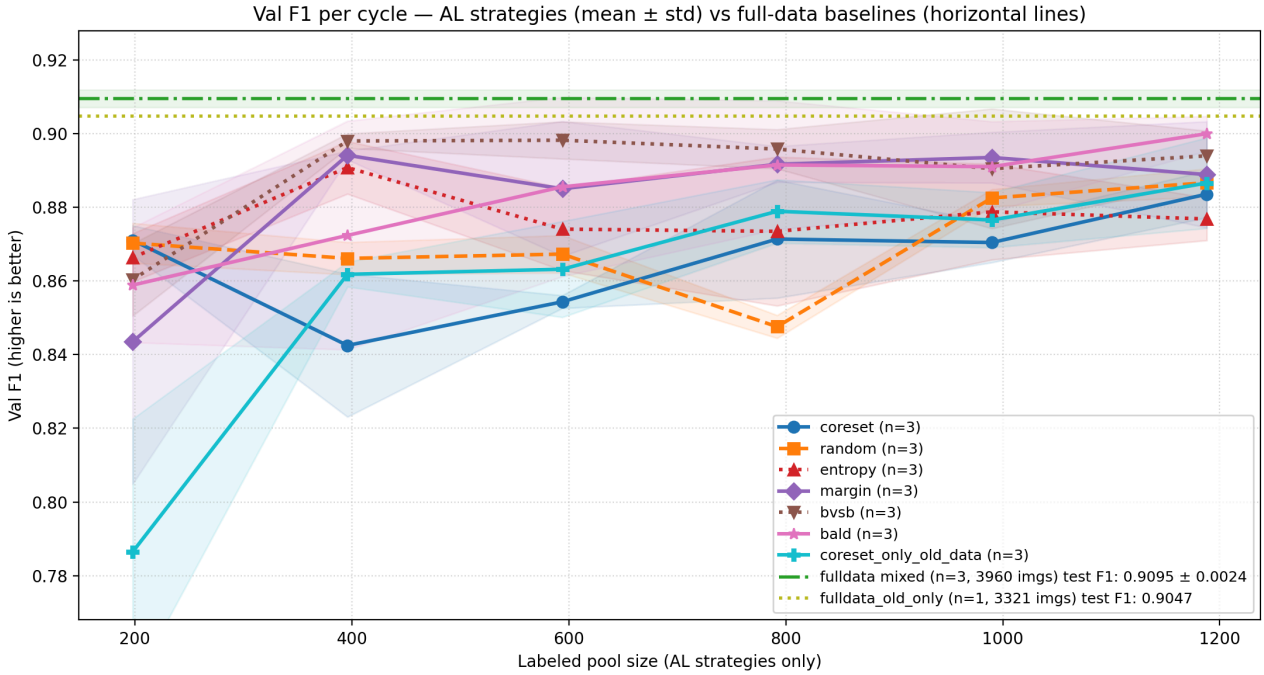


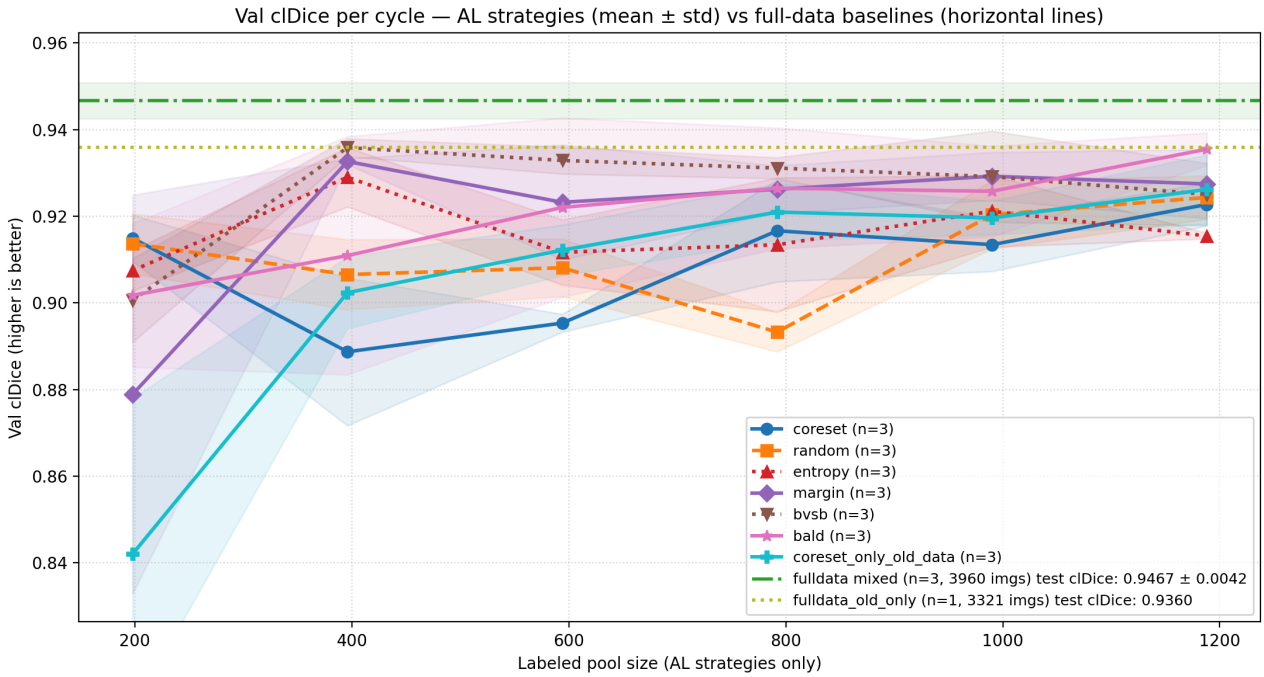
8.2 Val curves per cycle — mean ± std bands

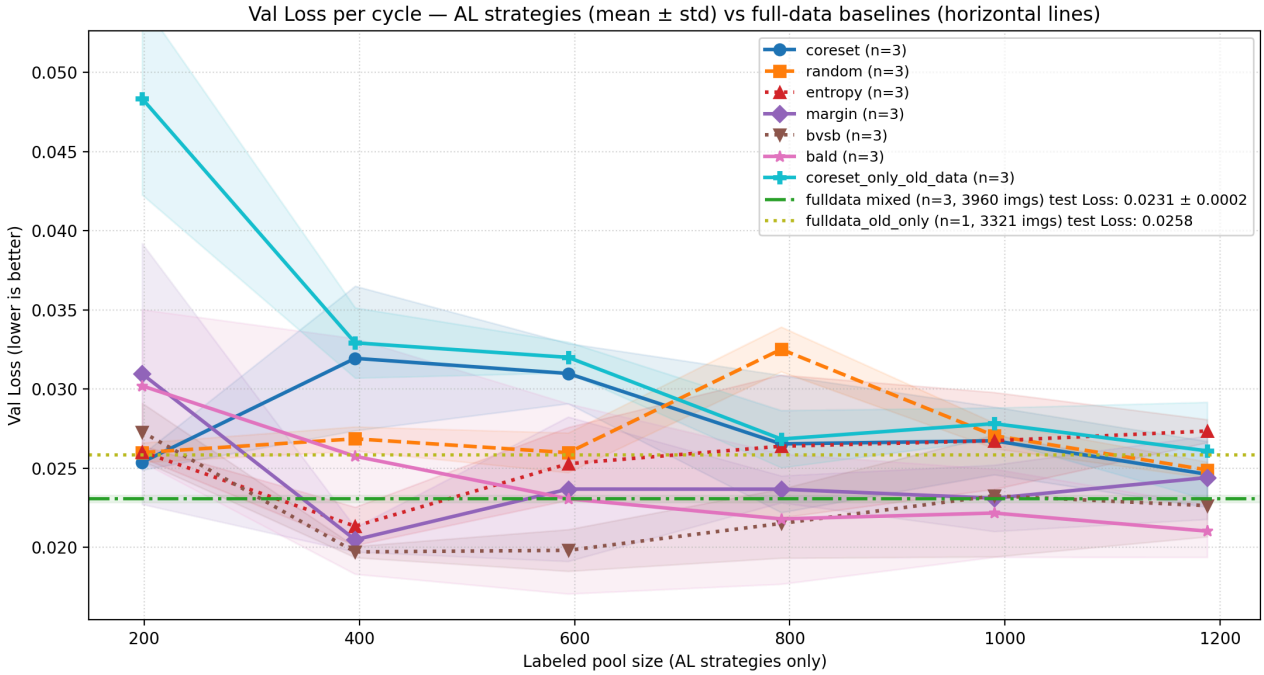
Pool size grows 198 → 396 → 594 → 792 → 990 → 1188 across cycles 0..5. The fulldata test value is shown as a horizontal reference line.

Phase 1:

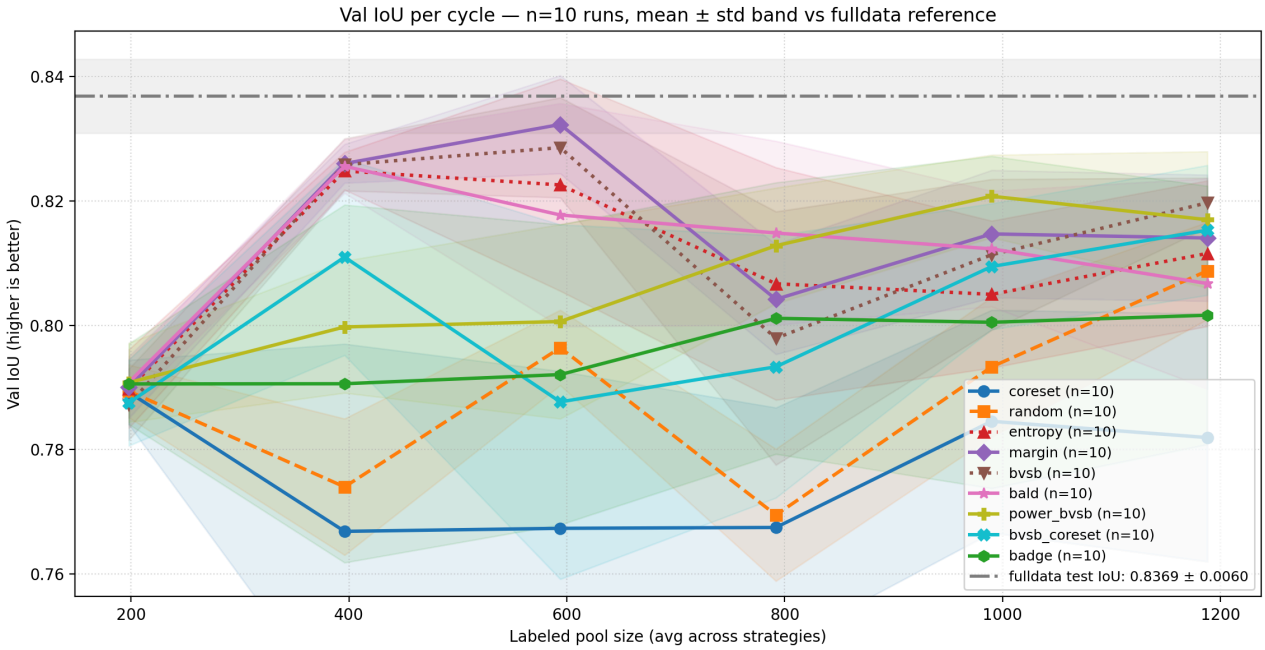

Val IoU per cycle — AL strategies (mean ± std) vs full-data baselines (horizontal lines)
Val IoU (higher is better)
Labeled pool size (AL strategies only)
0.85
0.80
0.75
0.70
0.65
200
400
600
800
1000
1200
coreset (n=3)
random (n=3)
entropy (n=3)
margin (n=3)
bvsb (n=3)
bald (n=3)
coreset_only_old_data (n=3)
fulldata mixed (n=3, 3960 imgs) test IoU: 0.8386 ± 0.0046
fulldata_old_only (n=1, 3321 imgs) test IoU: 0.8323

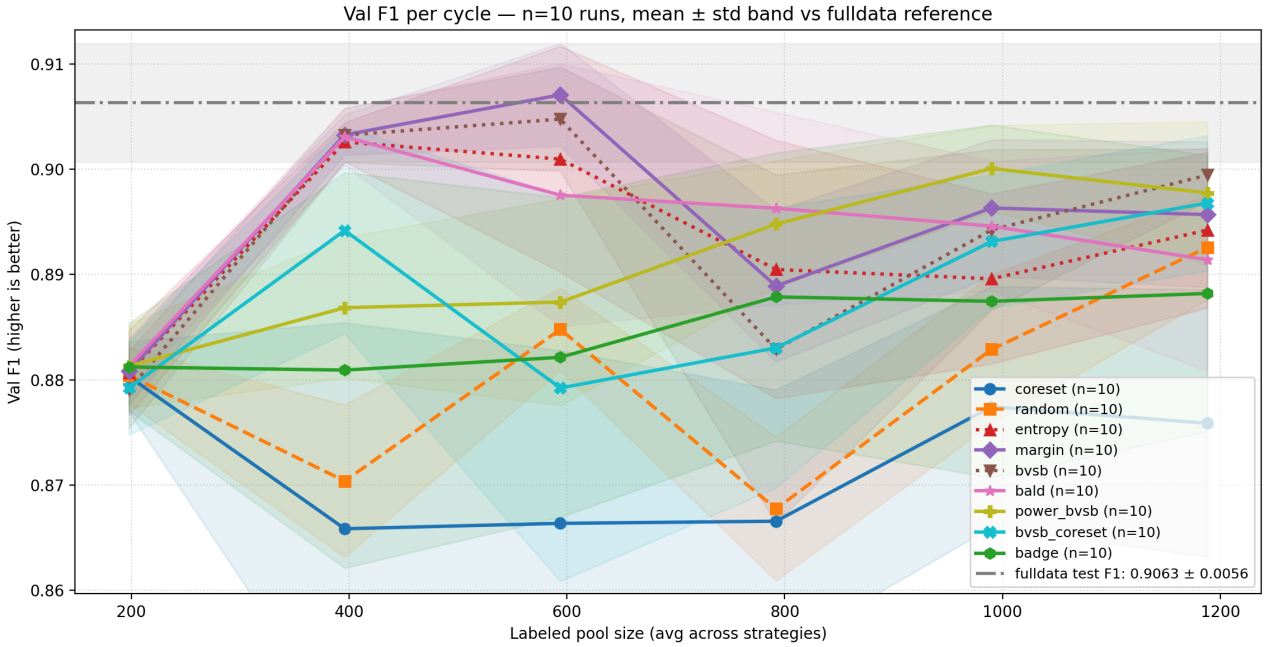

Val F1 per cycle — AL strategies (mean ± std) vs full-data baselines (horizontal lines)
Val F1 (higher is better)
Labeled pool size (AL strategies only)
0.92
0.90
0.88
0.86
0.84
0.82
0.80
0.78
200
400
600
800
1000
1200
coreset (n=3)
random (n=3)
entropy (n=3)
margin (n=3)
bvsb (n=3)
bald (n=3)
coreset_only_old_data (n=3)
fulldata mixed (n=3, 3960 imgs) test F1: 0.9095 ± 0.0024
fulldata_old_only (n=1, 3321 imgs) test F1: 0.9047

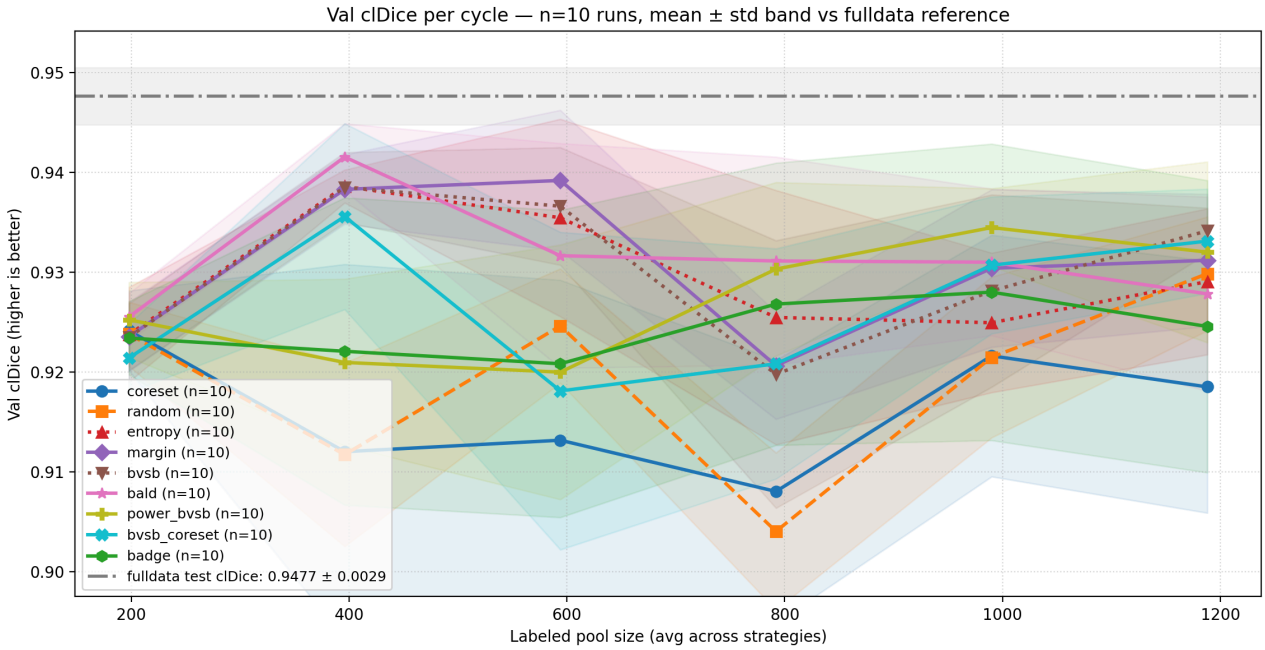


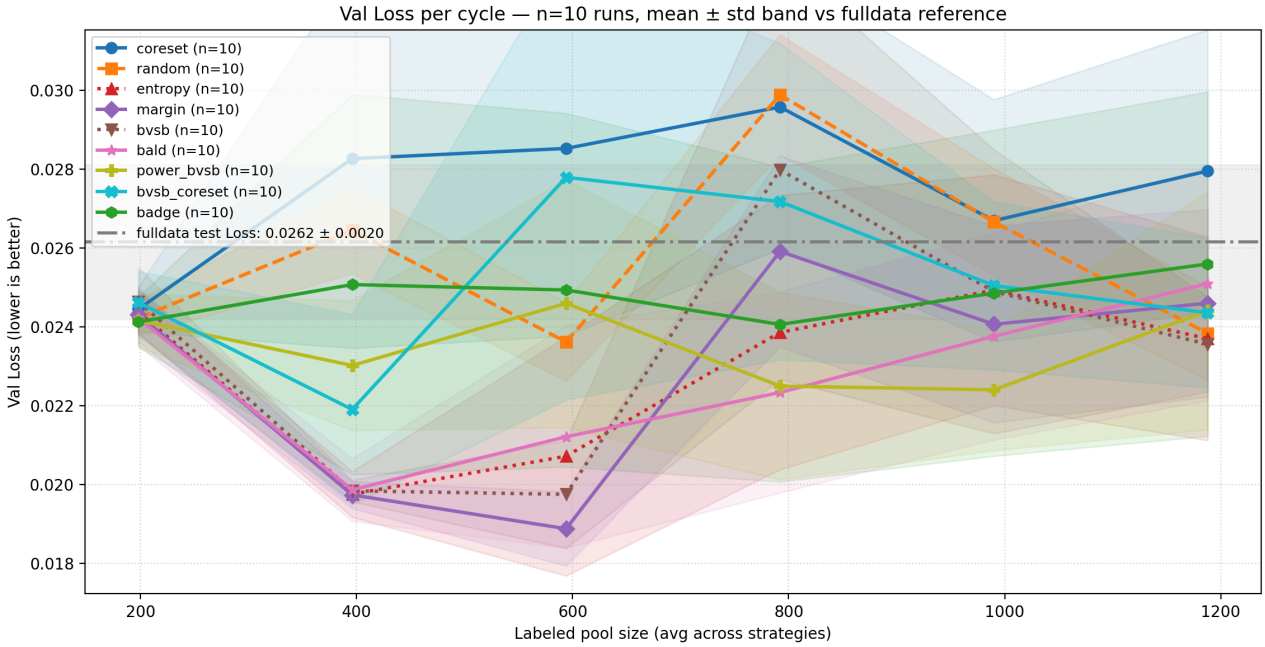


Phase 2 (all 9 strategies on one chart):


Val IoU per cycle — n=10 runs, mean ± std band vs fulldata reference
Val IoU (higher is better)
0.84
0.82
0.80
0.78
0.76
200
400
600
800
1000
1200
Labeled pool size (avg across strategies)
coreset (n=10)
random (n=10)
entropy (n=10)
margin (n=10)
bvsb (n=10)
bald (n=10)
power_bvsb (n=10)
bvsb_coreset (n=10)
badge (n=10)
fulldata test IoU: 0.8369 ± 0.0060


Val F1 per cycle — n=10 runs, mean ± std band vs fulldata reference
Val F1 (higher is better)
0.91
0.90
0.89
0.88
0.87
0.86
200
400
600
800
1000
1200
Labeled pool size (avg across strategies)
coreset (n=10)
random (n=10)
entropy (n=10)
margin (n=10)
bvsb (n=10)
bald (n=10)
power_bvsb (n=10)
bvsb_coreset (n=10)
badge (n=10)
fulldata test F1: 0.9063 ± 0.0056



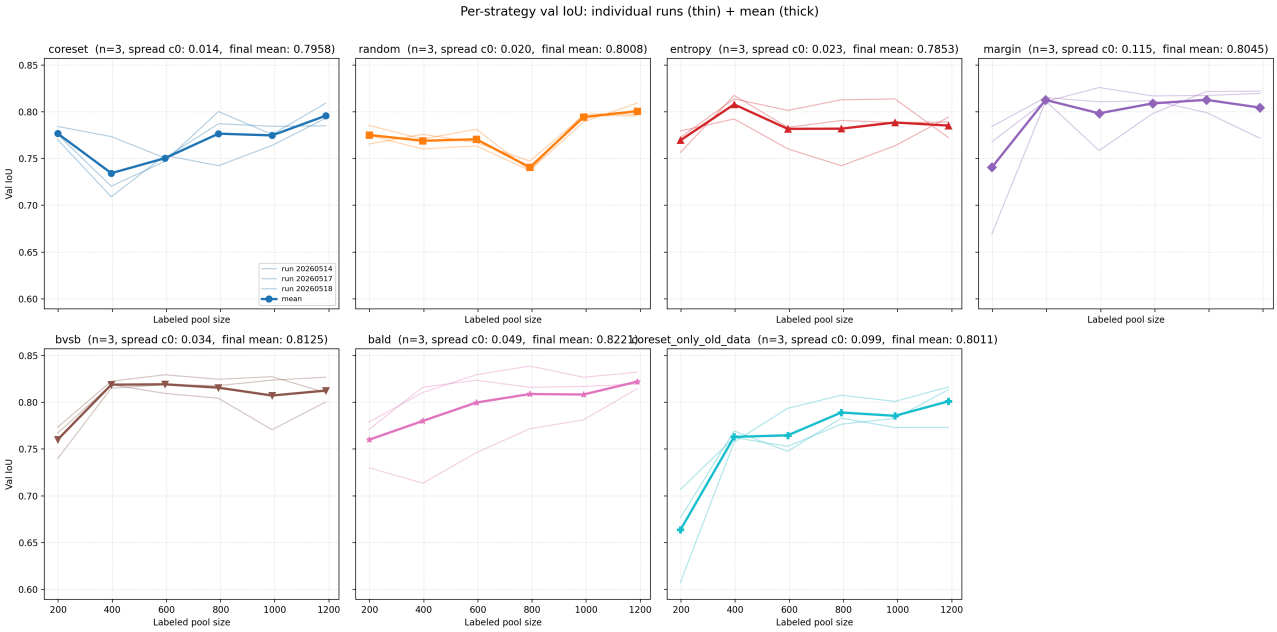


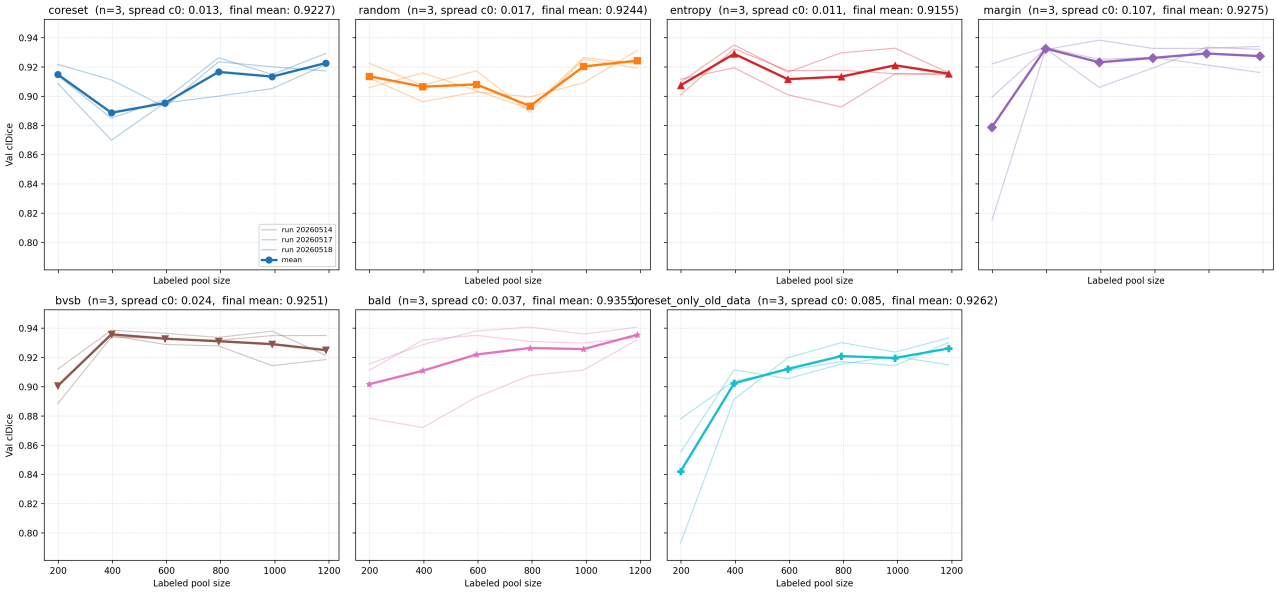
Convergence speed differs sharply by strategy in Phase 2: `margin` and `bvsb` are already near peak at cycle 1; `bald` *peaks* at cycle 1 (mean val IoU 0.8256) and then declines monotonically to 0.8067 by cycle 5, so its best checkpoint almost always comes from an early cycle; `random` genuinely needs all six cycles (best cycle = 5 in 9/10 runs); `coreset` often never beats its cycle-0 model (best val IoU at cycle 0 in 4/10 runs).

8.3 Per-strategy reproducibility (individual runs + mean)

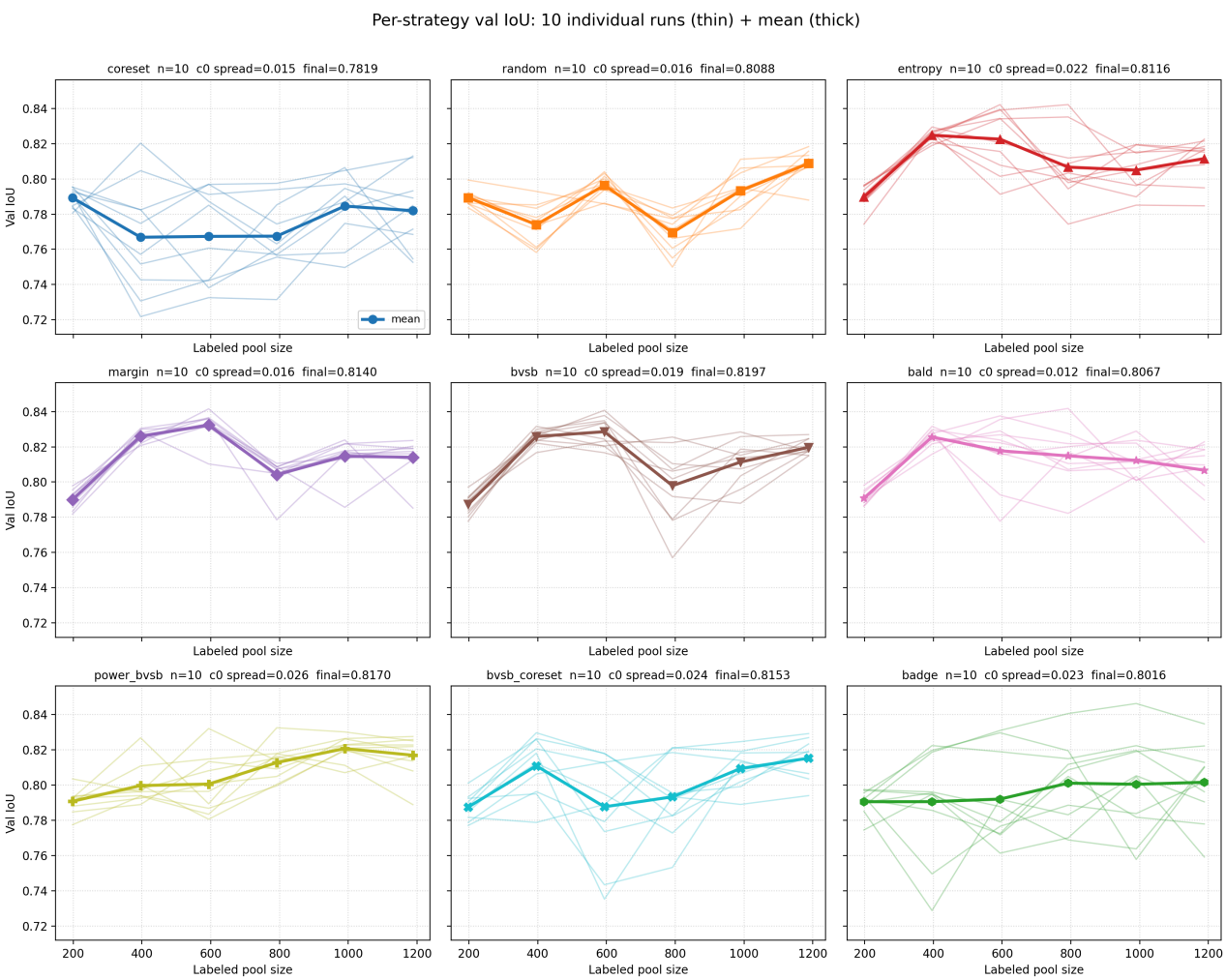
Each panel shows the n individual run curves (thin) plus the mean curve (thick). Useful for spotting outlier seeds.

Phase 1 (n=3, 2×4 grid — 7 panels, incl. the `coreset_only_old_data` ablation):





Phase 2 (n=10, 3×3 grid, one panel per strategy):



The Phase 1 `margin` panel shows the seed-2 outlier (run `20260517_005354`: cycle-0 IoU 0.6698 vs 0.7679 / 0.7844 on the other two — spread 0.1146); the Phase 2 panel doesn't have that outlier — `margin` cycle-0 spread across n=10 is only 0.0161.

Cycle-0 val IoU spread per strategy across all runs in each phase — the direct measure of training non-determinism (§9.1):

strategy	Phase 1 spread (n=3)	Phase 1 mean ± std	Phase 2 spread (n=10)	Phase 2 mean ± std
coreset	0.0141	0.7768 ± 0.0058	0.0148	0.7892 ± 0.0052
random	0.0198	0.7753 ± 0.0081	0.0157	0.7894 ± 0.0042
entropy	0.0229	0.7696 ± 0.0096	0.0222	0.7898 ± 0.0063
margin	**0.1146**	0.7407 ± 0.0506	0.0161	0.7900 ± 0.0050
bvsb	0.0336	0.7601 ± 0.0146	0.0194	0.7873 ± 0.0057
bald	0.0489	0.7600 ± 0.0214	0.0121	0.7908 ± 0.0037
coreset_only_old_data	**0.0992**	0.6638 ± 0.0415	—	—
fulldata_old_only	0.0000 (n=1)	0.8264	—	—
power_bvsb	—	—	0.0260	0.7908 ± 0.0064
bvsb_coreset	—	—	0.0241	0.7876 ± 0.0069
badge	—	—	0.0229	0.7906 ± 0.0065

Two observations from this table:

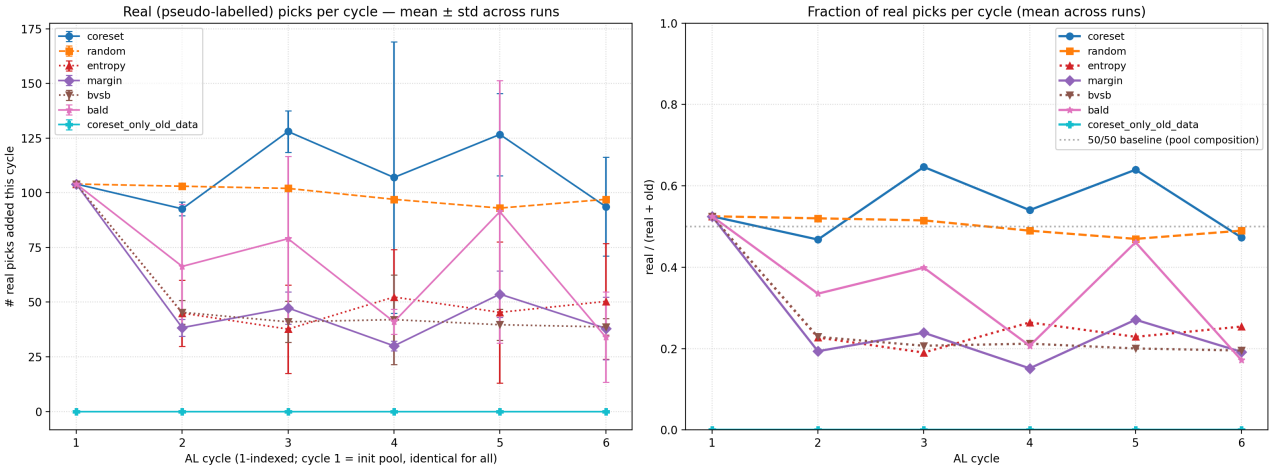
1. **Phase 1 was severely affected by the `margin` seed-2 outlier (spread 0.1146) and the `coreset_only_old_data` outlier (0.0992).** Both of those drove the §9.1 "non-determinism is a real problem" warning. Phase 2 confirms what was implied: with n=10 those tails do not recur (max spread 0.026, on `power_bvsb`).
2. **Phase 2 cycle-0 means are uniformly higher than Phase 1** (0.787–0.791 vs 0.760–0.777 with the margin-outlier run excluded). The cause is unmeasured — note that BATCH_SIZE=8 *halves* the optimizer steps per epoch (§9.6), so "larger batches train faster per epoch" does not work as a step-count argument, and the §9.8 scoring fix cannot explain cycle 0 (no scoring has happened yet). Open question.

8.4 Picks composition — which pool does each strategy reach for?

Recall the training pool is 50/50 `real` (noisy pseudo-labels) / `old` (clean). Eval pool is 100% `old`.

Phase 1, cycles 1–5 picks summed, mean across n=3:

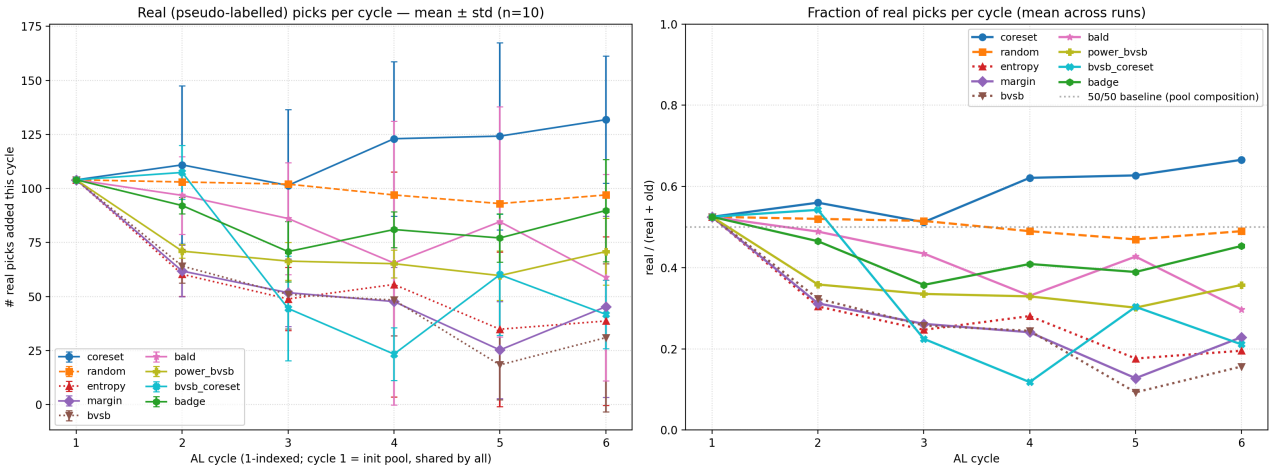
strategy	real picks	old picks	real fraction
coreset	548.0	442.0	**55.4 %** (above pool ratio)
random	492.0	498.0	49.7 % (≈ pool ratio)
bald	311.7	678.3	31.5 %
entropy	230.7	759.3	23.3 %
margin	207.3	782.7	20.9 %
bvsb	206.7	783.3	20.9 %



(Plot x-axes are 1-indexed with cycle 1 = the shared init pool, so the tables' pick cycles 1–5 appear at plot cycles 2–6.)

Phase 2, cycles 1–5 picks summed, mean across n=10:

strategy	real picks	old picks	real fraction
coreset	591.3	398.7	**59.7 %** (above pool ratio, more than Phase 1)
random	492.0	498.0	49.7 % (≈ pool ratio)
badge	410.8	579.2	41.5 %
bald	391.8	598.2	39.6 % (Phase 1 was 31.5 %)
power_bvsb	333.1	656.9	33.6 %
bvsb_coreset	277.2	712.8	28.0 %
entropy	238.4	751.6	24.1 %
margin	231.9	758.1	23.4 %
bvsb	212.9	777.1	21.5 %



In both phases the AL test ranking and the real-fraction column are **inversely correlated, but imperfectly** (Spearman ρ ≈ −0.88 in Phase 2, ≈ −0.66 in Phase 1). coreset (60 %) and badge (42 %) sit at the bottom of the ranking; bvsb / margin / entropy (21–24 %) sit at the top. `bald` **is the standing counterexample:** it picks the most `real` of any uncertainty method (39.6 % in Phase 2, 31.5 % in Phase 1) yet ranks 4th of 10 in Phase 2 and 1st in Phase 1 — picking noisy data does not by itself sink a strategy, so

the noise-concentration mechanism below is a tendency, not a law. **Bonus Phase 2 fact:** `bvsb_fixmatch` picks 37.7 % `real` (cycles 1–7 summed across n=5) — higher than supervised BvSB because at the small budget the model is less confident and the BvSB ratio sees more borderline pixels in the harder `real` images.

The k-center mechanism (§8.5.2) generalises to the new diversity-aware methods. Anything that pulls picks toward atypical regions of feature space — k-center (coreset), Gumbel-perturbed BvSB (power_bvsb), uncertainty × k-center (bvsb_coreset), gradient-embedding k-means++ (badge) — ends up reaching into the noisier `real` distribution. Pure uncertainty avoids it implicitly because the model is more uncertain on the cleaner `old` images it sees less of during training.

8.5 Why coreset loses — the outlier hypothesis + n=3 ablation

8.5.1 What pick composition shows

See §8.4 — coreset is **above** the 50/50 pool ratio in both phases, picking ~55–60 % `real`. The four uncertainty methods sit at 21–40 % (bald the highest — and also the strategy least hurt by it, see §8.4).

8.5.2 The mechanism

k-center greedy maximises distance from the currently labeled set. The `real` pool is the noisier, more visually atypical source (variable lighting, occlusions, angles). Those are exactly the points that sit furthest from the rest of the feature space, so k-center reaches for them first. The val/test set is 100 % `old`, so training more on noisy pseudo-labels hurts at evaluation time.

Uncertainty methods do the opposite implicitly — the model is more uncertain on clean `old` (a distribution it effectively sees less of during training even with the 50/50 mix), so they preferentially flag `old` and do *de facto* domain adaptation toward the test distribution.

8.5.3 The Phase 1 ablation: coreset on old-only pool (n=3)

Re-ran coreset with `n_real=0` in `CombinedDataset` — only the clean `old` images are available for selection (all 3321 of them). Absolute budget per cycle was held at 198 picks/cycle (`INIT_BUDGET = BUDGET_PER_CYCLE = 198/3321 ≈ 0.0596`), so the labeling cost matches the original coreset run.

Runs:

- `al_run_20260521_180410` (seed 1, May 21 — the original)
- `al_run_20260523_030634` (seed 2, May 23 — added)
- `al_run_20260523_134454` (seed 3, May 23 — added)

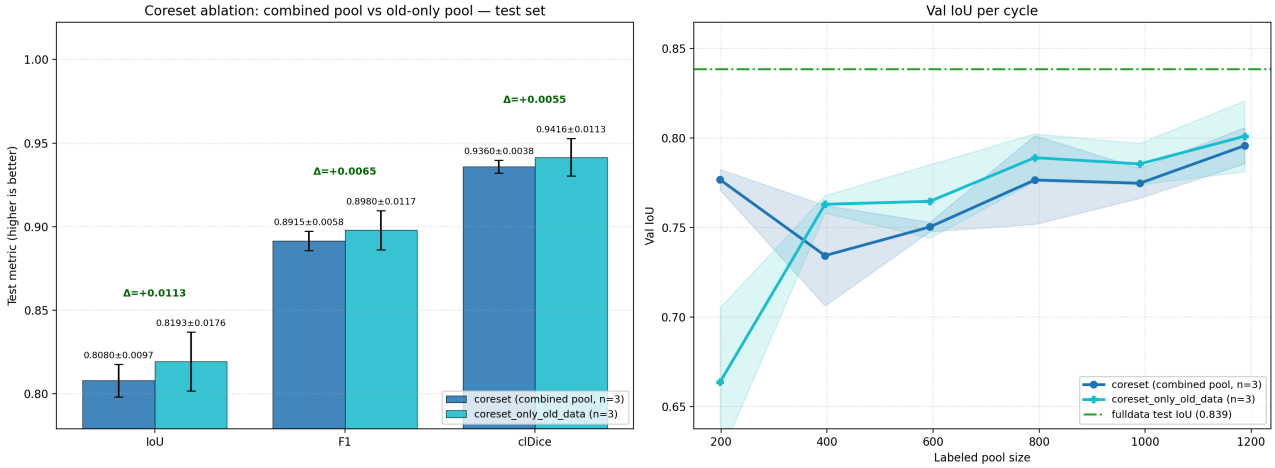
Per-seed TEST IoU: 0.8302, 0.7945, 0.8332 → **0.8193 ± 0.0176**.

metric	coreset (combined pool, n=3)	coreset_only_old_data (n=3)	Δ
IoU	0.8080 ± 0.0097	0.8193 ± 0.0176	+0.0113 ↑
F1	0.8915 ± 0.0058	0.8980 ± 0.0117	+0.0065 ↑
clDice	0.9360 ± 0.0038	0.9416 ± 0.0113	+0.0055 ↑
Loss	0.0249 ± 0.0020	0.0266 ± 0.0031	+0.0017 ↓ worse

What changed vs the n=1 version of this finding:

- The IoU gain shrinks from **+2.2pp → +1.1pp**.
- The std of the ablated coreset (±0.0176) is now **larger than the effect size**, and ~1.8× the combined-pool coreset's std.
- Loss flipped sign: combined-pool coreset has **lower** loss than old-only (0.0249 vs 0.0266) — old-only is more confidently wrong on hard examples.
- Cycle-0 val IoU is alarmingly noisy: 0.7069 / 0.6766 / 0.6077 in seed order s1/s2/s3 (spread 0.099) — much wider than any main strategy at cycle 0. The `warn_only=True` "fix" did not actually make the ablation reproducible.

Revised conclusion. Pool composition probably still helps coreset a little (+1.1pp mean IoU), but the effect is **within ~1σ of noise** and the original n=1 "ablation strongly supports the trap hypothesis" framing was over-claimed. The ablated coreset still places between `entropy` and `random` on the ranking, so coreset isn't fixed by removing the noisy pool — k-center is still doing worse than uncertainty methods on this task.



8.6 The other Phase 1 ablation: fulldata on old-only pool

Natural companion to §8.5.3: if `real` is so noisy that it's hurting coreset, shouldn't dropping it from the training pool help every strategy? In particular, shouldn't fulldata on **clean labels only** beat fulldata on the mixed pool?

Run: `al_run_20260522_190442` (single seed, n=1). Trained for 40 epochs on all 3321 `old` images (no `real`); evaluated on the same holdout test set.

metric	fulldata (mixed pool, 3960, n=3)	fulldata_old_only (3321 old, n=1)	Δ
IoU	0.8386 ± 0.0046	**0.8323**	**−0.0063 ↓ worse**
F1	0.9095 ± 0.0024	0.9047	−0.0048 ↓ worse
clDice	0.9467 ± 0.0042	0.9360	−0.0107 ↓ worse
Loss	0.0231 ± 0.0002	0.0258	+0.0028 ↓ worse

The clean-only fulldata is WORSE than the mixed-pool fulldata, across every metric. The `real` pseudo-labels — even though model-generated and noisy, even though the test set is 100 % clean — **add useful supervision when combined with the clean labels**. The network is learning something from them: probably distribution coverage (different lighting, angles, occlusions that the `old` tiles don't show), not pixel-perfect rail localisation.

This **contradicts the strongest reading of §8.5** — namely "the noisy pool was the problem and AL routed around it." The truer story is more nuanced:

- For **AL strategies**, picking heavily from `real` is bad (concentrates training on noisy supervision and underrepresents the test distribution).
- For **fulldata**, including all the `real` images **alongside** all the `old` images is good (the model has enough clean signal to anchor the learning, and the noisy data adds diversity).

The two findings together suggest that **the issue with coreset isn't that `real` images are inherently harmful — it's that coreset disproportionately picks them when the clean labels are scarce.** Same data, different effect depending on how much of it dominates the training signal.

8.7 The low-budget regime: where AL wins big (Phase 1, n=3)

The main 6-strategy runs use init = 5 % + 5 %/cycle, so by cycle 1 every strategy has 10 % of labels and the curves are already flat. To map the regime where AL actually matters, we ran a separate experiment with **init = 99 (2.5 %) + 14 selection cycles × 39 picks (1 %/cycle) = 645 labels final (~16 % of 3960)**. (`NUM_CYCLES = 15` in the notebook is the total number of training cycles; the first is the init pool.) Only `random` and `bvsb` were run. n=3 seeds:

- `al_run_20260524_002008` (seed 1 — the original n=1 result)
- `al_run_20260525_153057` (seed 2 — `nb_low_budget_s2.ipynb`, May 25)
- `al_run_20260526_211123` (seed 3 — `nb_low_budget_s3.ipynb`, May 26)

budget	random TEST IoU	bvsb TEST IoU	bvsb − random
645 (~16 %, n=3)	**0.7926 ± 0.0012**	**0.8332 ± 0.0052**	**+0.0406 ± 0.0064**
1188 (30 %, main n=3)	0.8171 ± 0.0055	0.8391 ± 0.0045	+0.0220

Reference: fulldata @3960 (100 %, n=3) = 0.8386 ± 0.0046.

What survives the n=3 update (see §7.6.4):

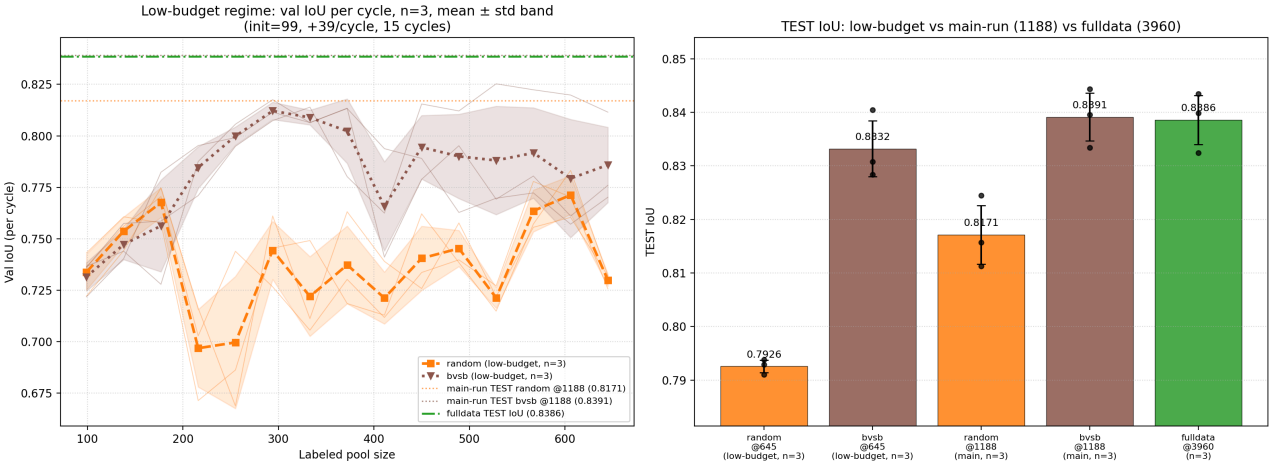
1. **AL gains do concentrate where labels are scarce.** Δ over random is +4.06pp ± 0.64pp at 16 %, about 1.85× the +2.20pp at 30 %.
2. **Random is remarkably stable at 16 %** (std 0.0012).
3. **BvSB does NOT match fulldata at 16 % with n=3.** Headline change: 0.8332 vs 0.8386, ~1σ below. The n=1 0.8404 "matches" was the lucky seed.
4. **BvSB at 16 % < BvSB at 30 %** by ~0.6pp — extra label budget buys small but non-zero performance.

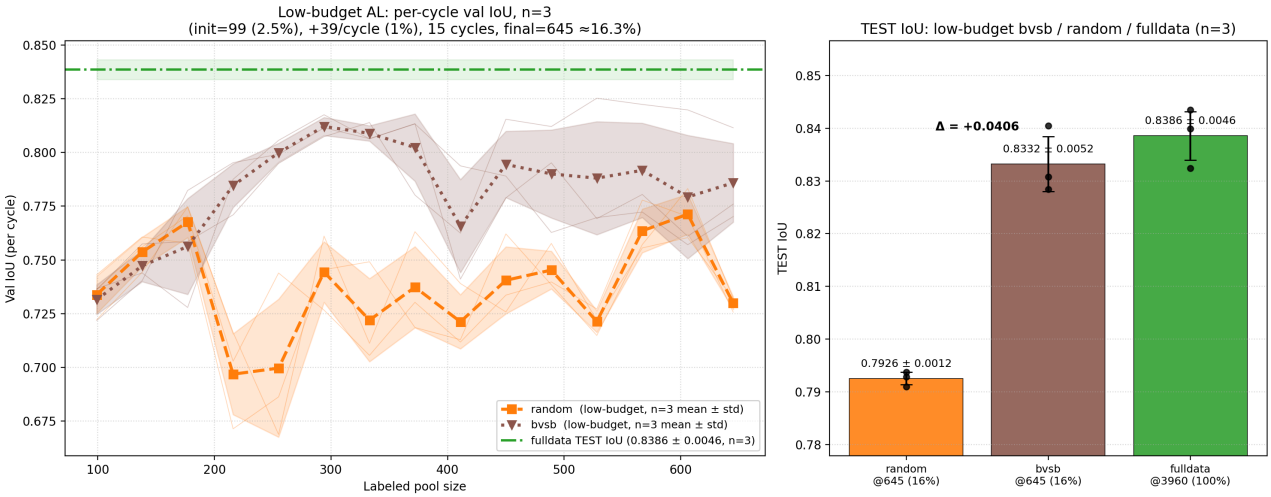
Caveats:

- Per-cycle val IoU is jagged for `random` (swings 0.67–0.78 within a seed). BvSB's per-cycle val IoU is less jagged but not monotonic — see plots below.
- All three seeds use the same `set_seed(42 + cycle)` machinery → the determinism issue from §9.1 affects them in similar ways. std=0.0052 for BvSB and 0.0012 for random reflect both seed variation and CUDA non-determinism between seeds; not cleanly separable.
- All three seeds use the post-fix scoring pipeline (§9.8), so the apples-to-apples comparison vs main BvSB @1188 (pre-fix scoring) is still confounded. Phase 2 main bvsb (n=10) uses **the same post-fix scoring**, so the cleanest "BvSB at 16 % vs BvSB at 30 %" comparison going forward is Phase 1 low-budget (n=3, 0.8332) vs Phase 2 main BvSB (n=10, 0.8416), still BATCH_SIZE-confounded.

- The picks composition for the low-budget runs isn't reported here.

Plots — comprehensive view (with main-run and fulldata reference lines) plus the focused view:





8.8 NEW (Phase 2): diversity-aware strategies — power_bvsb, bvsb_coreset, badge

The three new strategies were added to test the standard AL-literature intuition that **uncertainty × diversity > pure uncertainty**. None of them helps on this task:

strategy	TEST IoU (n=10)	Δ vs bvsb	Δ vs random
bvsb	0.8416 ± 0.0048	—	+0.0206
power_bvsb	0.8324 ± 0.0066	−0.0092	+0.0113
bvsb_coreset	0.8303 ± 0.0069	−0.0113	+0.0092
badge	0.8256 ± 0.0109	−0.0161	+0.0045
random	0.8211 ± 0.0067	−0.0206	—

`power_bvsb` and `bvsb_coreset` improve on random with statistical support (Welch t = 3.6 and 2.9 at n=10); `badge`'s +0.45pp does **not** reach significance (t = 1.0, p ≈ 0.3). All three lose to plain BvSB decisively (|t| = 3.4–4.0). The mechanism — picks-composition § 8.4 — is the same one that breaks pure coreset: any move that pulls picks toward atypical regions of feature space concentrates training on the noisy `real` pool, which the test set doesn't share.

Per-strategy interpretation:

- **`power_bvsb`** (β = 2.0). Adding Gumbel-top-K stochasticity to the BvSB ranking. Picks-composition jumps from 21.5 % real (bvsb) to 33.6 % real (power_bvsb). The stochastic perturbation lets it explore points BvSB would have ignored — and those points are disproportionately in the noisy pool. β → ∞ would recover vanilla BvSB; here β = 2.0 is enough perturbation to hurt.
- **`bvsb_coreset`** (BvSB top-3K → k-center). The diversity step trims away near-duplicates among the top BvSB picks. But at 30 % budget on a 3960-pool, redundancy isn't the bottleneck — noise concentration is. Picks-composition 28.0 % real (between bvsb and power_bvsb).
- **`badge`** (gradient embedding + k-means++). The canonical uncertainty × diversity benchmark from Ash et al. 2020 — and the worst performer of the three. Picks-composition 41.5 % real, nearly as bad as coreset. The gradient-embedding direction `(p - one_hot) ⊗ feat` is large in magnitude on points where the model is both uncertain and far from its current training distribution — i.e. exactly the noisy `real` outliers. k-means++ then explicitly spreads picks across that gradient space, amplifying the over-pick effect.

The unified read: on this dataset, in this regime (30 % budget, thin-class segmentation, noisy half-pool, 100 %-`old` evaluation), diversity is solving the wrong problem. Pure uncertainty wins because it implicitly avoids the noisy distribution by being more uncertain on the cleaner distribution it sees less of during training. Anything that introduces an explicit diversity term reverses that selection pressure and hurts.

These three sit in the Phase 2 ranking and Δ plots in §8.1 — there's no separate per-strategy figure for them.

8.9 NEW (Phase 2): FixMatch SSL — first probe

Why we ran it. Mittal et al. 2019 ("Parting with Illusions about Deep Active Learning") argues that most AL gains evaporate once you add a strong SSL objective. Our Phase 1 low-budget finding (BvSB beats random by +4.06pp at 16 % labels) is exactly the kind of claim Mittal's framework would stress-test. So the question is: **does BvSB still beat random under FixMatch?**

What we ran. BvSB only, no random+FixMatch (intentional deferral — that's the next experiment, FIXMATCH_NOTES §11). 8 cycles, init = 99 + 7×39 → 372 labels (9.4 % of 3960), τ = 0.95, λ_u = 1.0, μ = 1.0 (unlabeled batch 4, labeled batch 4 — **not** the Phase 2 supervised batch of 8; see §9.6). n = 5 seeds.

What we got.

comparator	TEST IoU	budget	n
FixMatch BvSB @372	**0.8218 ± 0.0045**	9.4 %	5
Phase 2 supervised BvSB @1188	0.8416 ± 0.0048	30 %	10
Phase 1 supervised BvSB low-budget @645	0.8332 ± 0.0052	16 %	3
Phase 2 fulldata @3960	0.8369 ± 0.0060	100 %	10
Phase 1 supervised random low-budget @645	0.7926 ± 0.0012	16 %	3

(Batch-size note: the FixMatch and Phase 1 rows trained at labeled batch 4; the Phase 2 supervised rows at batch 8 — the §9.6 caveat applies within this table.)

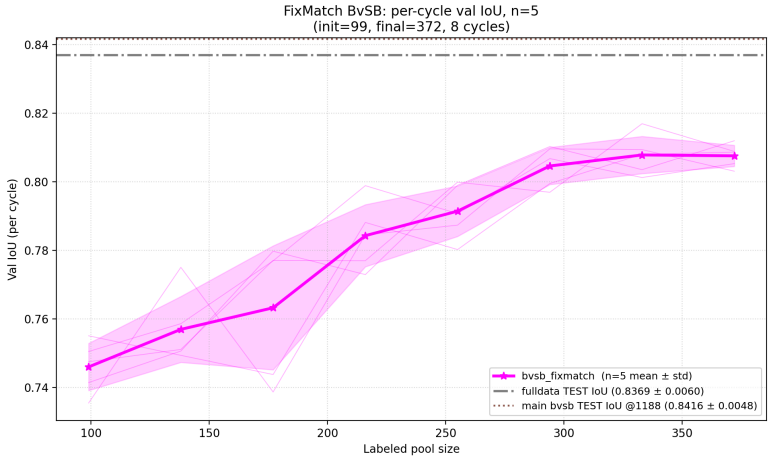
The honest reading. We cannot conclude "FixMatch helps" or "FixMatch hurts" without a matched random+FixMatch baseline at the same 9.4 % / 8-cycle configuration. What we *can* observe:

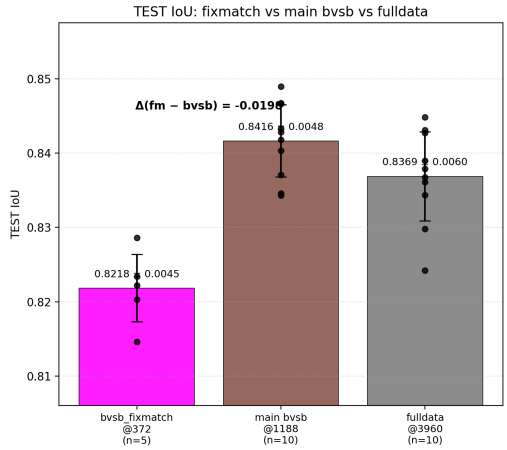
- FixMatch BvSB at 9.4 % labels = 0.8218, which is **−1.5pp below** Phase 2 fulldata (0.8369) and **−2.0pp below** Phase 2 supervised BvSB at 30 % (0.8416). SSL did not push it to fulldata performance with this much fewer labels.
- It is **+2.9pp above** Phase 1 random at 16 % labels (0.7926), but at a *lower* budget (9.4 % vs 16 %) and under a different objective (FixMatch vs supervised). So that gap mixes strategy, budget, and objective (labeled batch size happens to match — both at 4).
- The picks-composition for `bvsb_fixmatch` is **37.7 % real** across cycles 1–7 (n=5) — significantly higher than supervised bvsb's 21.5 %. The small budget and the consistency-regularised model produce a flatter / less-confident probability distribution, so BvSB sees more borderline pixels in the noisier `real` images and reaches for them.

The matrix that would actually answer the Mittal question:

	supervised	**FixMatch**
BvSB	Phase 1 low-budget @645 (n=3) ✓	Phase 2 @372 (n=5) ✓
random	Phase 1 low-budget @645 (n=3) ✓	**not run**

Plus matched budget (currently FixMatch is at 9.4 % while low-budget supervised is at 16 %). The cell-28 random+FixMatch run is the next thing to schedule.





9. Known issues & caveats

9.1 Training non-determinism — partially "fixed", still a problem

`set_seed()` sets cuDNN's deterministic flag, but several CUDA ops (`ConvTranspose2d` backward, `F.interpolate`, scatter-style loss reductions) use atomicAdd which is order-non-deterministic. Empirically:

- **Within a single Phase 1 run**, cycle-0 IoU differs across strategies even though they share `init_labeled` and `set_seed(42)`. Spread of 5.5 / 10.9 / 4.5pp across the three main runs (in run order s1/s2/s3).
- **Across the 3 Phase 1 main runs**, cycle-0 IoU spread per strategy ranges from 0.014 (coreset) to **0.115 (margin)**. Margin's spread is a potential outlier — one run (seed 2) gave 0.6698 vs 0.7679 /

0.7844 on the others.

- **Across the n=3 `coreset_only_old_data` ablation runs**, cycle-0 IoU spread is **0.099** (0.7069 / 0.6766 / 0.6077 in seed order) — and this is on the notebooks that include the supposed "fix" below.
- **Across Phase 2's n=10 main runs**, cycle-0 IoU spread per strategy is much tighter — **0.012–0.026** across all 9 strategies, with the worst case `power_bvsb` at 0.026 and `bald` at 0.012. The margin outlier did not recur with n=10.

Fix that was applied for Phase 1 May 22–24 follow-ups and inherited by Phase 2:

```
os.environ['CUBLAS_WORKSPACE_CONFIG'] = ':4096:8'
torch.use_deterministic_algorithms(True, warn_only=True)
```

`warn_only=True` is the catch: it allows non-deterministic ops to silently run with a warning instead of raising. `F.interpolate(bilinear)` backward and a few scatter-style ops in the loss don't have deterministic CUDA kernels, so they run non-deterministically under the flag. The fix doesn't make the runs strictly reproducible. To actually fix this would require either (a) `warn_only=False` plus deterministic replacements for every offending op (some don't exist), or (b) accepting non-determinism and averaging over many seeds. Phase 2 is closer to (b).

Final TEST IoU std (0.003–0.018) is much smaller than cycle-0 std because picking "best cycle" implicitly does a max-over-noisy reduction.

9.2 Per-batch IoU averaging

`validate_one_epoch` and `test_model` call `MulticlassJaccardIndex` per batch and take the arithmetic mean of the per-batch IoUs. For thin-class binary segmentation this is biased — batches with no class-1 pixels in either GT or prediction give degenerate per-batch IoU values. The correct approach is `metric.update(...)` per batch and `metric.compute()` once at the end, which accumulates intersection / union.

This affects every reported number, but **affects all strategies equally**, so relative rankings are still trustworthy. **Applies to both phases.**

9.3 Best-epoch = final-epoch within a cycle

`train()` runs 40 epochs and returns whatever's at epoch 40. There's no per-epoch validation, so the "best checkpoint" for a strategy is the final-epoch model of whichever cycle had the highest val IoU. The actual within-cycle peak may be at epoch 25 or 35; we never see it. Applies to both phases. Fix: add per-epoch (or every-k-epoch) validation — cheap at 989 val images — so each cycle checkpoints its true best epoch.

9.4 Initial pool seed drift

The notebook initially used `np.random.default_rng(96)` to shuffle indices for the init pool (and `dump_init_pool.py` still does). At some point the notebook was edited to `np.random.default_rng(42)`, which produces a *different* init pool (92 real / 106 old instead of 104 real / 94 old). All Phase 1 main runs used seed=96 and were mutually consistent. The notebooks built for the May 22–24 follow-up runs and Phase 2 explicitly set `INIT_POOL_SEED = 96` to lock this — no drift in the reported results, but worth keeping pinned permanently.

9.5 `coreset_only_old_data` cycle-0 spread is huge

When the ablation was n=1 (May 21), the cycle-0 IoU was 0.7069 — nothing to compare against. With n=3, the three runs gave 0.7069 / 0.6766 / 0.6077 (seed order s1/s2/s3). That's a 0.099 spread, **7× wider** than the Phase 1 main-run coreset cycle-0 spread (0.014).

Possible causes:

- The augmentation-fix edit (using `train_eval_dataset` with `val_transform` for scoring, added May 22) changed the embedding geometry that coreset operates on, making the first picks more sensitive to model init noise.
- The old-only pool (3321 images, no real) has different feature-space structure than the mixed pool, making k-center less stable.
- The "fix" in §9.1 isn't actually deterministic.

This is why the §8.5.3 effect size has shrunk so much: the noise floor went up, not the signal. Worth investigating before publishing the ablation.

9.6 NEW — BATCH_SIZE drift between phases (4 → 8)

Phase 1 used `BATCH_SIZE = 4` everywhere; Phase 2 changed it to 8 in `train_clearml.ipynb` cell 13 **on the server copy** (the local copy linked elsewhere in this report still has the Phase 1 value of 4). Intentional, for wall-clock per cycle. Consequences:

- All Phase 2 absolute numbers are not drop-in replacements for Phase 1 numbers. See §7.5 for the per-strategy shifts. The largest shifts: `entropy` +2.1pp, `margin` +1.0pp. The smallest: `bald`, `fulldata` (−0.17pp, i.e. slight downward drift within their own σ) and `coreset` (+0.17pp) — a three-way tie at display precision; `bvsb` moved +0.25pp.
- Per-cycle batches changes from 50 → 25 at cycle 0 (pool=198). The cosine scheduler `T_0 = 6 * len(train_loader)` therefore has a different period in absolute step count between the two phases — a subtle interaction with the training dynamics that we did not measure.
- Phase 2 FixMatch does **not** keep the labeled batch at 8 — the FixMatch notebook reverts to labeled batch 4 with `FIXMATCH_BATCH_SIZE_UNLABELED = 4` (μ = 1.0). FixMatch runs are therefore batch-matched to *Phase 1*, not to the Phase 2 supervised runs; every FixMatch-vs-Phase-2 comparison (§7.4, §8.9) carries this confound.

For any future paper writeup, treat Phase 1 and Phase 2 as two separate experiments at two different batch sizes that happen to have the same ranking structure, not as a single dataset.

9.7 NEW — ClearML files server returns HTTP 405 on uploads

Verified empirically during Phase 2 setup: `task.upload_artifact(...)` and `Logger.report_image(local_path=...)` both register metadata in the ClearML API correctly, but the binary upload to the files server endpoint returns **HTTP 405 Method Not Allowed**. Behaviour was identical with `output_uri=False`, `output_uri=True`, and explicit `Logger.set_default_upload_destination("http://clearml.infra.niias/files")`. Repro probe (logged in chat): a test artifact uploaded to ClearML showed metadata + a URL, but `HEAD` / `GET` on that URL returned 404, and the SDK debug log showed:

```
clearml.storage - ERROR - Exception encountered while uploading
Failed uploading object .../artifacts/.../tmp...pkl
to http://clearml.infra.niias/files (405): <!doctype html>
```

Consequences for Phase 2:

- No `.pkl` artifacts in ClearML — everything lives in `runs/{label}_{RUN_ID}/` on the dev box (and now in `/media/samsepi/data/server_runs/`).
- No debug-sample JPEGs in ClearML Debug Samples panel — the per-cycle `result_images/.../cycle_NN.jpg` files exist on disk but were never uploaded.
- Scalars (TB metrics mirrored to ClearML) **do** work — those go through the events endpoint, which is not the files endpoint.

This is a server-side fileserver / proxy configuration issue (probably GET-only behind nginx), not a bug in our code. To fix server-side, talk to the infra team about PUT/POST handling on `clearml.infra.niias/files`.

9.8 Augmentation-fix edit changed the experimental setup mid-stream

Between the original 3 Phase 1 main runs and the May 22–24 follow-ups, the notebooks were edited so that scoring (coreset embeddings, entropy / margin / bvsb / bald) reads the unlabeled pool through a **second dataset object with `val_transform` (no augmentations)** instead of the augmented `train_dataset`. This is methodologically correct (the previous behaviour was scoring on a different random augmentation each cycle, adding noise to distances and uncertainty scores), but it means the May 22–24 results are not strictly comparable to the May 14–18 results for any strategy that uses scoring:

- `coreset_only_old_data` **seeds 2 and 3** (May 23) use the fixed (un-augmented) scoring; **seed 1** (May 21) used the pre-fix augmented scoring. The n=3 mean ± std in §8.5.3 therefore mixes methodologies.
- The Phase 1 low-budget runs' `bvsb` use fixed scoring; the Phase 1 main `bvsb` used augmented scoring. The Phase 1 low-budget headlines are partially confounded by this.
- All Phase 2 runs use the post-fix scoring — so internal Phase 2 comparisons are clean, and cross-phase comparisons need the same caveat for any strategy that scores.
- `random` is unaffected because it doesn't compute uncertainty / embeddings on the unlabeled pool (it just `rng.choice`s).

For `random` the fix is irrelevant. For coreset it matters most. The §8.5.3 "Δ shrunk" finding therefore confounds two changes: more seeds + scoring methodology. A clean rerun of `coreset_only_old_data` and the Phase 1 main `coreset` strategy with the fixed scoring is the right next step.

---

10. File index

10.1 `plots_comparison/` (this directory, Phase 1)

file	what it is
`EXPERIMENT_REPORT.md`	this document
`REVIEW_PROMPT.md`	self-contained prompt for independently auditing this report
`REVIEW_FINDINGS.md`	results of the June 2026 independent audit (its findings are folded into this revision)
`reproducibility_table.txt`	mean ± std test metrics + cycle-0 spread + coreset head-to-head + fulldata head-to-head + low-budget summary, Phase 1 numbers
`summary.csv`	long-format Phase 1: one row per (run, strategy, cycle, kind, metric). Includes the 15-cycle low-budget rows as `bvsb_low_budget` / `random_low_budget`. 2833 lines.
`test_bars_{IoU,F1,clDice,Loss}.png`	Phase 1 ranked horizontal bars, mean ± std, individual run dots, fulldata reference line. Includes `coreset_only_old_data` (n=3) and `fulldata_old_only` (n=1).
`test_delta_vs_random_{...}.png`	Phase 1 bars of (strategy − random) per metric
`test_delta_vs_fulldata_{...}.png`	Phase 1 bars of (strategy − fulldata) per metric
`val_curves_{IoU,F1,clDice,Loss}.png`	Phase 1 per-cycle mean ± std curves, one line per strategy + fulldata baseline
`val_per_strategy_{IoU,clDice}.png`	Phase 1 2×4 grid: 3 individual run curves + thick mean curve per strategy
`picks_composition.png`	Phase 1 mean real / old picks per cycle (left) and real-fraction per cycle (right), per strategy
`coreset_ablation.png`	Phase 1 focused 2-panel ablation: bars of test IoU/F1/clDice with Δ annotations + per-cycle val IoU curve. coreset (combined pool, n=3) vs coreset_only_old_data (n=3)
`low_budget_curves.png`	Phase 1 2-panel comprehensive view (n=3): per-cycle val IoU mean ± std bands for low-budget bvsb / random (15 cycles), with thin per-seed traces and reference lines for main-run bvsb / random / fulldata; right panel = TEST IoU bars comparing low-budget @645 vs main-run @1188 vs fulldata @3960
`low_budget_only.png`	Phase 1 focused 2-panel view of just the low-budget regime: per-cycle val IoU mean ± std bands for bvsb / random with thin per-seed traces and a single fulldata TEST IoU reference line; right panel = TEST IoU bars (n=3 with per-seed dots) for low-budget bvsb / random / fulldata only

10.2 `plots_comparison/server_runs/` (Phase 2)

Copied from `/media/samsepi/data/server_runs/plots/` for portability of this report.

file	what it is
`reproducibility_table.txt`	Phase 2 mean ± std test metrics (n=10) + cycle-0 spread + FixMatch summary (n=5) + head-to-head
`summary.csv`	Phase 2 long-format: one row per (run, kind_run, strategy, cycle, train_size, kind, metric, value); includes both main and fixmatch rows
`test_bars_{IoU,F1,clDice,Loss}.png`	Phase 2 ranked horizontal bars (n=10) for all 9 strategies + fulldata reference
`test_delta_vs_random_{...}.png`	Phase 2 Δ vs random
`test_delta_vs_fulldata_{...}.png`	Phase 2 Δ vs fulldata
`val_curves_{IoU,F1,clDice,Loss}.png`	Phase 2 per-cycle mean ± std curves, one line per strategy, all 9 strategies on one chart
`val_per_strategy_IoU.png`	Phase 2 3×3 grid: 10 individual run curves + thick mean per strategy
`picks_composition.png`	Phase 2 mean real / old picks per cycle (n=10), 9 strategies
`fixmatch_curves.png`	Phase 2 FixMatch BvSB per-cycle val IoU (n=5 with mean ± std band + per-seed traces) + TEST IoU bars vs main BvSB @1188 + fulldata

10.3 Other (off-tree)

- `/media/samsepi/data/server_runs/` — original copy of the per-run pkl/pth archive (~25 GB, 10 main + 5 fixmatch directories), plus the `plot_server_runs.py` aggregator and an `EXPERIMENTS.md` directory description.
- On the dev box: `/workspace/ultralytics1/SyzdykovN/active_learning/runs/` is the canonical copy.
- On the dev box: `logs/{RUN_ID}/` TensorBoard scalar files (1.8 MB total) — not transferred to this report directory; pkls were sufficient.

---

11. TL;DR (combining both phases)

- **Use BvSB (or margin) in production.** Phase 2 n=10: margin (0.8434), entropy (0.8420), bvsb (0.8416), bald (0.8398) are all within 0.004 IoU of each other — statistically tied. BvSB has the practical edge: no MC-Dropout passes (vs BALD), tight std (0.0048), and behaves consistently across both phases. Margin is the new top point estimate in Phase 2 and has the lowest variance of any strategy (std 0.0031).
- **The "AL matches fulldata at 30 % labels" headline got stronger.** Phase 1 (n=3): BALD/BvSB within ±1σ of fulldata. Phase 2 (n=10): margin/entropy/bvsb/bald all sit **above** fulldata. The loss column is the cleanest version (Phase 2): AL methods at ~0.018, fulldata at ~0.026 — **~30 % lower test loss with 70 % fewer labels** (the Phase 1 loss gap is a more modest ~11 %). AL is performing data curation against the noisy `real` pool.
- **AL advantage over random in low-budget regime is robust at n=3.** +2.20pp at 30 %, **+4.06pp ± 0.64pp at 16 %**. Every individual seed has Δ ≥ +3.4pp (worst seed +3.46pp). Mittal's "random is hard to beat" applies above some budget; below it, AL clearly helps. (Phase 1 only — Phase 2 has no low-budget supervised runs.)

- **BvSB at 16 % labels is near but not equal to fulldata.** n=3 low-budget BvSB = 0.8332 ± 0.0052, vs fulldata 0.8386 ± 0.0046 — a 0.5pp gap, ~1σ below. The previous n=1 "matches fulldata at 16 %" was a lucky seed.
- **Three new diversity-aware strategies all underperform plain BvSB** (Phase 2 n=10): `power_bvsb` −0.92pp, `bvsb_coreset` −1.13pp, `badge` −1.61pp (badge's +0.45pp edge over random is itself not statistically significant). All three over-pick the noisy `real` pool relative to pure BvSB (33.6 % / 28.0 % / 41.5 % real vs BvSB's 21.5 %). On this task — thin-class segmentation, noisy half-pool, 100 %-clean evaluation — **uncertainty × diversity loses to pure uncertainty.**
- **Coreset is consistently the only AL strategy that loses to random.** Phase 1 −0.92pp, Phase 2 −1.14pp (the Phase 2 deficit is the statistically solid one). K-center reaches for the visually atypical (= noisier) `real` images first.
- **The "noisy pool is the only problem" story does NOT fully survive.** Two updates undermine it:
- `coreset_only_old_data` n=3 (Phase 1) gives +1.1pp (was +2.2pp at n=1), and the std (0.0176) is now larger than the effect.
- `fulldata_old_only` n=1 (Phase 1) is **worse** than mixed-pool fulldata by 0.6pp — the noisy `real` images aren't pure noise.

The truer story: coreset's loss is mostly **k-center being a poor AL criterion for thin-class segmentation on this dataset**, with a small additional cost from over-picking `real`. Generalised to Phase 2: all diversity-aware methods inherit the same over-pick problem.

- **FixMatch SSL did not close the small-budget gap on its own** (Phase 2, n=5). 0.8218 at 9.4 % labels vs Phase 2 supervised BvSB 0.8416 at 30 % (comparison also batch-confounded: FixMatch trains at labeled batch 4, supervised Phase 2 at 8 — §9.6). **No conclusion yet** — the matched random+FixMatch baseline (cell 28 of `nb_low_budget_fixmatch.ipynb`) hasn't been run. That's the Mittal test, and it's the obvious next experiment.
- **Methodological cleanup still wanted before paper-readiness:**
- Real determinism (`warn_only=False`, swap out non-deterministic ops or accept and average over many seeds).
- ~~n=3 of low-budget BvSB + random.~~ **Done** (§7.2 / §8.7).
- ~~n≥5 of main strategies.~~ **Done at n=10** (§7.3, Phase 2).
- n=3 of `fulldata_old_only`.
- Rerun `coreset` (combined pool) with the post-fix scoring at Phase 2 BATCH_SIZE=8, so its comparison to `coreset_only_old_data` is apples-to-apples on both axes.
- Run **random + FixMatch** at the matched 9.4 % / 8-cycle config (`nb_low_budget_fixmatch.ipynb` cell 28) to actually answer the Mittal question.
- Fix per-batch IoU averaging (§9.2).
- Per-epoch validation so "best cycle" is actually best (§9.3).
- Fix the ClearML 405 server-side or move to a working fileserver, so Phase 3+ doesn't have to rely on local `runs/` archives.